Lesson 02 Lesson-End Project

Implementing CI/CD for a Full Stack Application

Project agenda: To establish a CI/CD pipeline for a full stack application deployment on AWS Elastic Beanstalk

Description: Your team lead has asked you to establish a CI/CD pipeline on AWS for the streamlined deployment of a full-stack application. This pipeline will automate the deployment process of both frontend and backend parts of the application using AWS services, ensuring that code changes are automatically tested and deployed to production.

Tools required: AWS Management Console, AWS Elastic Beanstalk, AWS CodePipeline, AWS CodeCommit, and AWS CodeBuild

Prerequisites: An active AWS account

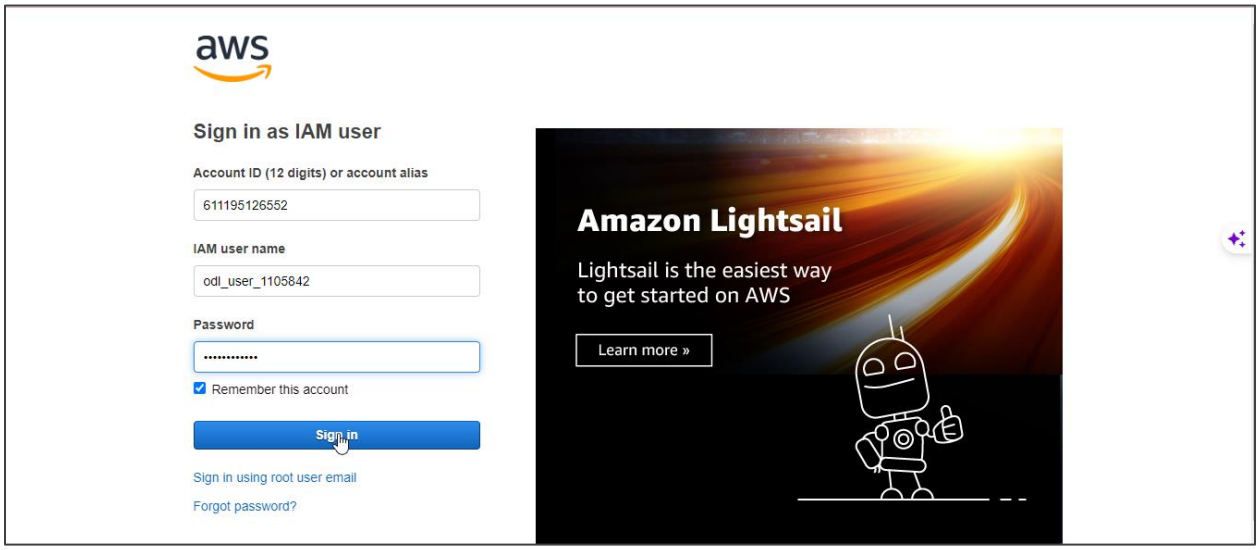
Expected deliverables: A functioning CI/CD pipeline on AWS for automated deployments

Steps to be followed:

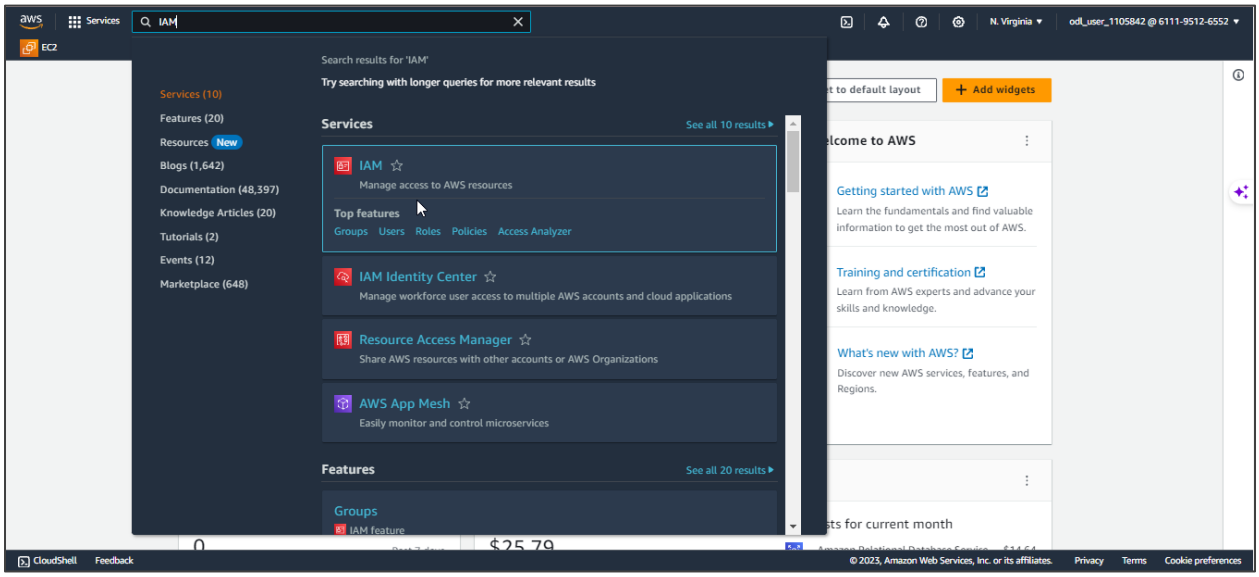
1. Set up an Elastic Beanstalk backend application
2. Set up a Node.js Beanstalk environment for the backend application
3. Set up an Elastic Beanstalk frontend application
4. Set up a Node.js Beanstalk environment for the frontend application
5. Set up the CodePipeline for the backend application
6. Set up the CodePipeline for the frontend application
7. Deploy the application

Step 1: Set up an Elastic Beanstalk backend application

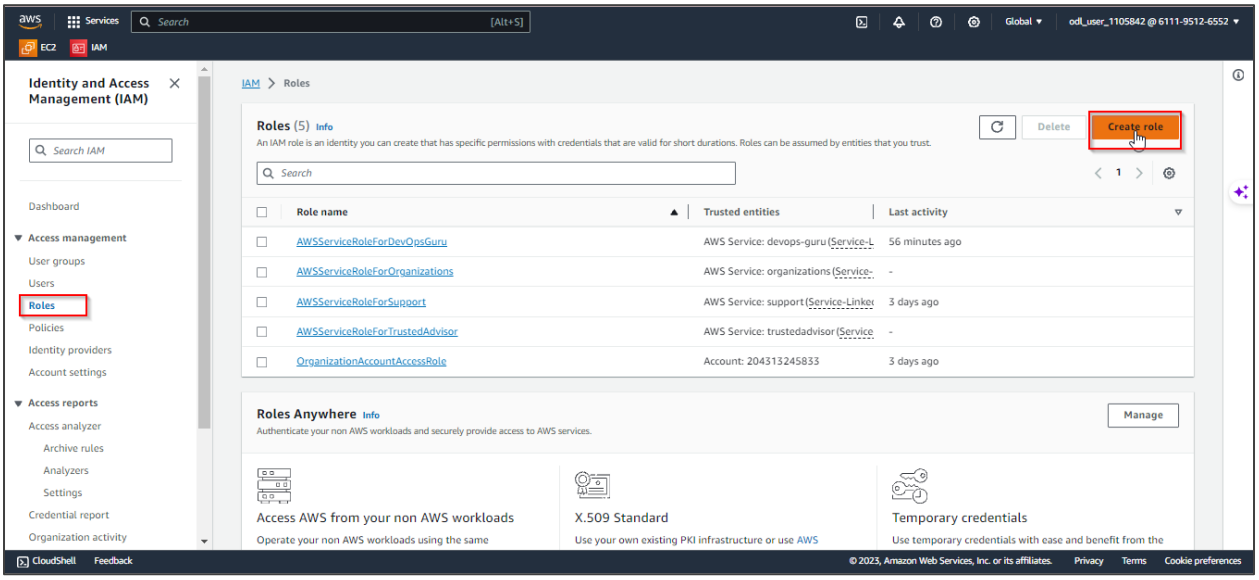
1.1 Sign in to the AWS Management Console using the provided AWS lab credentials



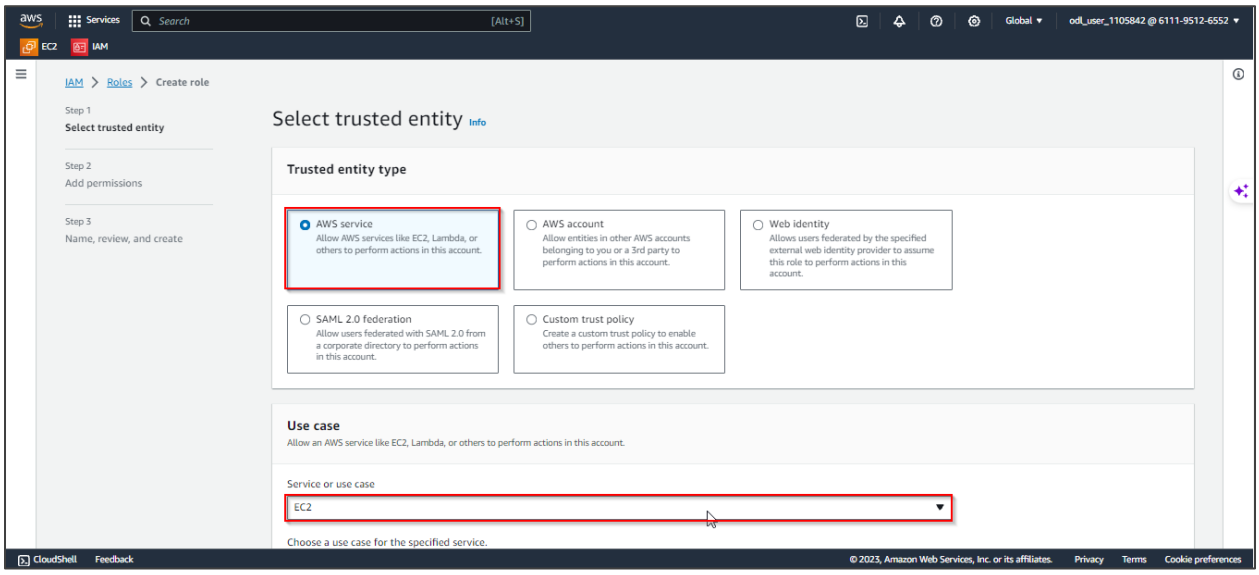
1.2 Navigate to the IAM Service to create an instance profile

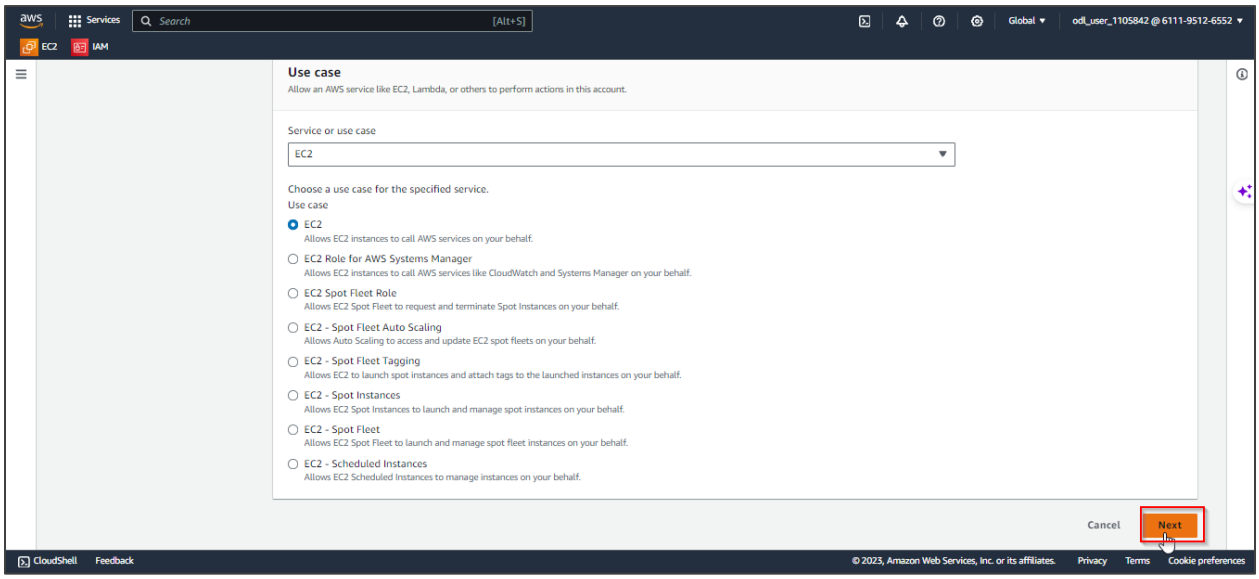


1.3 In the navigation pane, choose **Roles**, and then select **Create role**

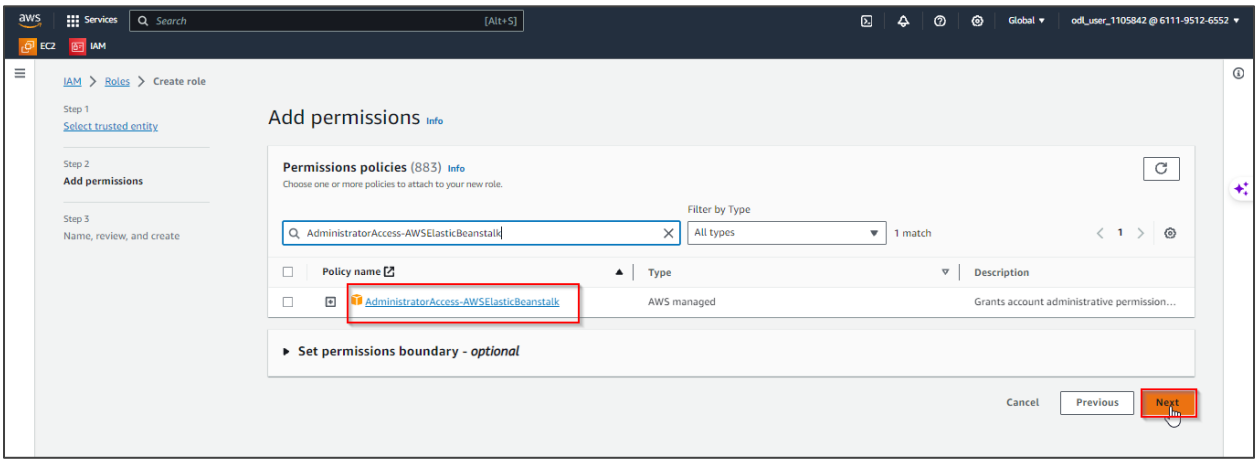


1.4 Choose **AWS service** as the entity type, select **EC2** as the service, and then click on **Next**

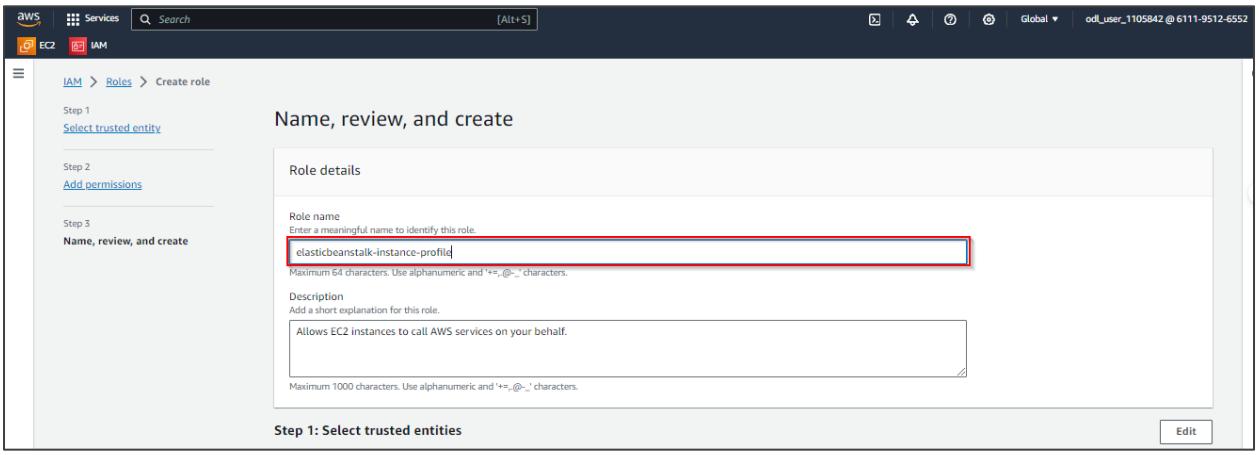


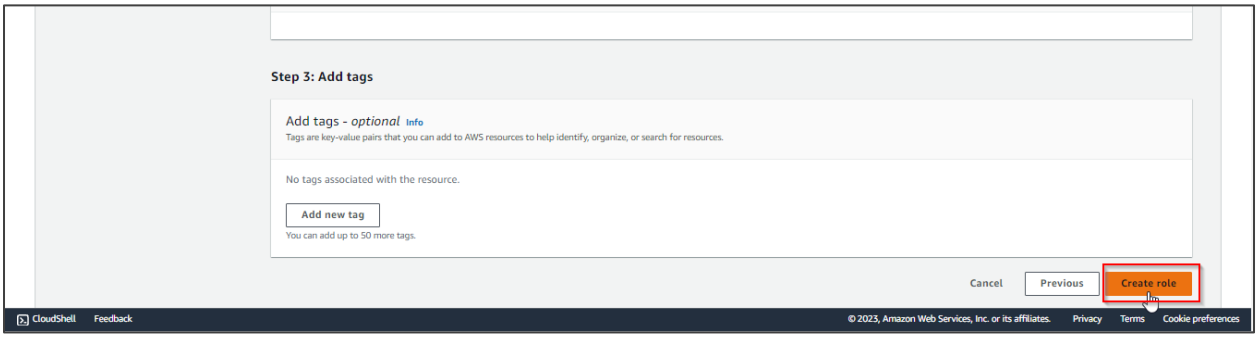


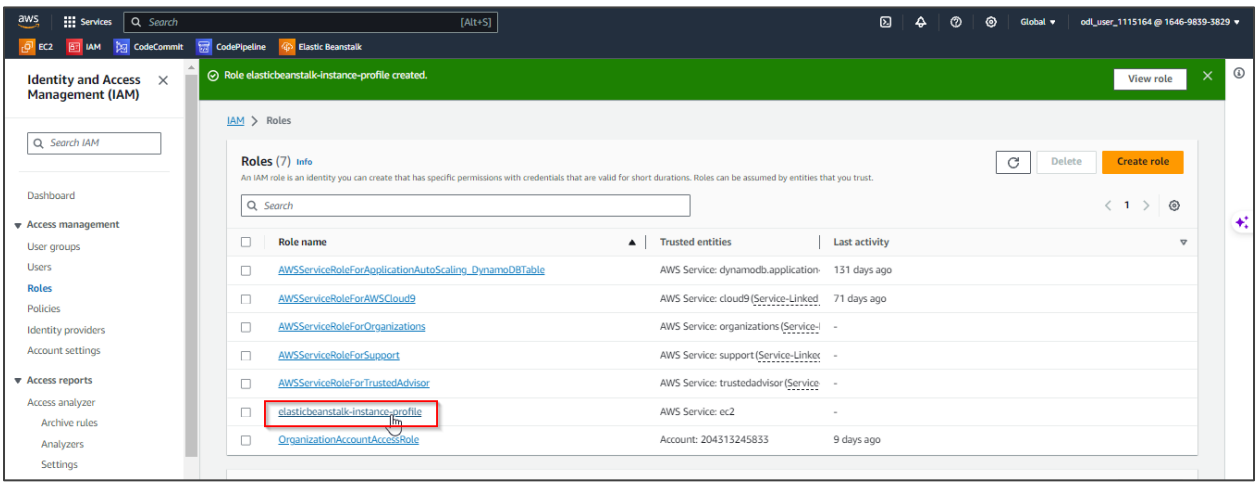
1.5 Select the **AdministratorAccess-AWSElasticBeanstalk** permission policy, then click on **Next**



1.6 Provide an appropriate name for the IAM role, review the details, and then click on **Create role**

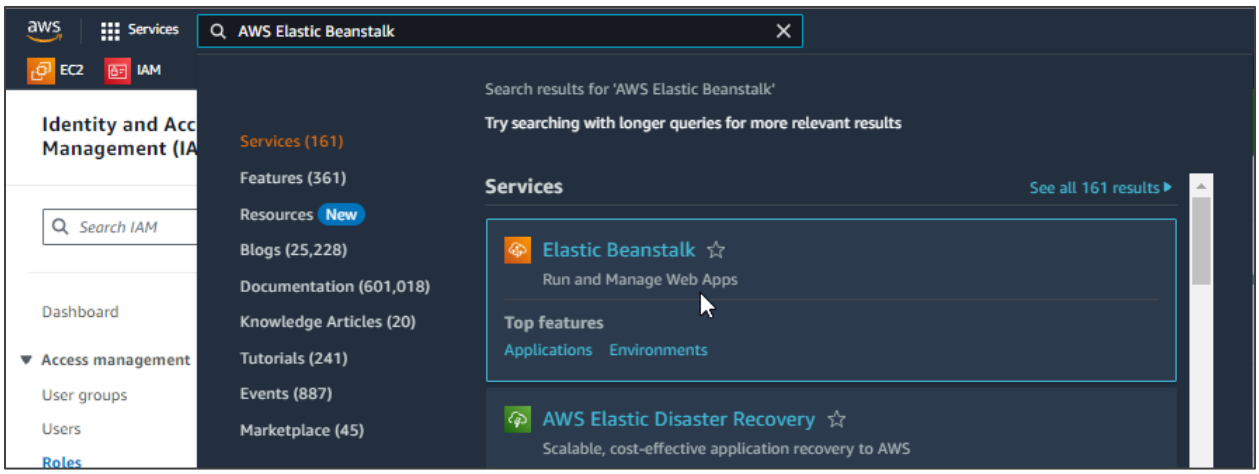




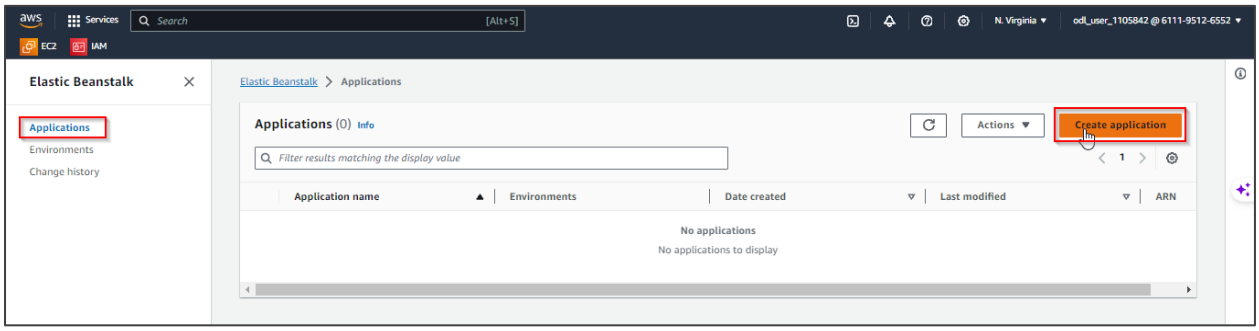


This action creates a new IAM role.

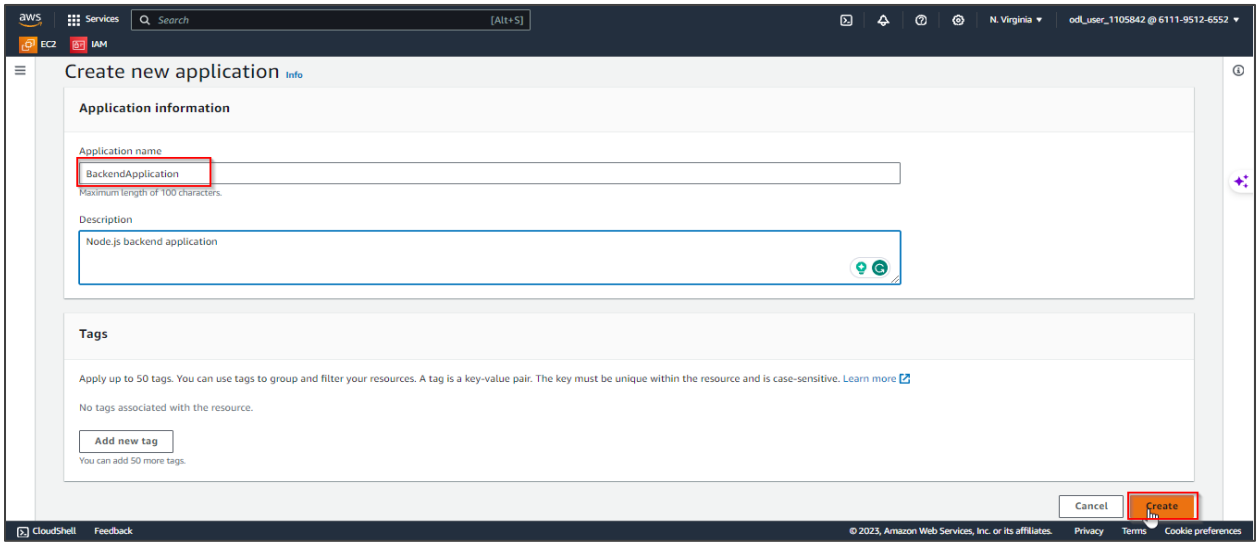
1.7 Navigate to the AWS **Elastic Beanstalk** service to create a new Node.js application



1.8 In the **Applications** section, click on **Create applications**

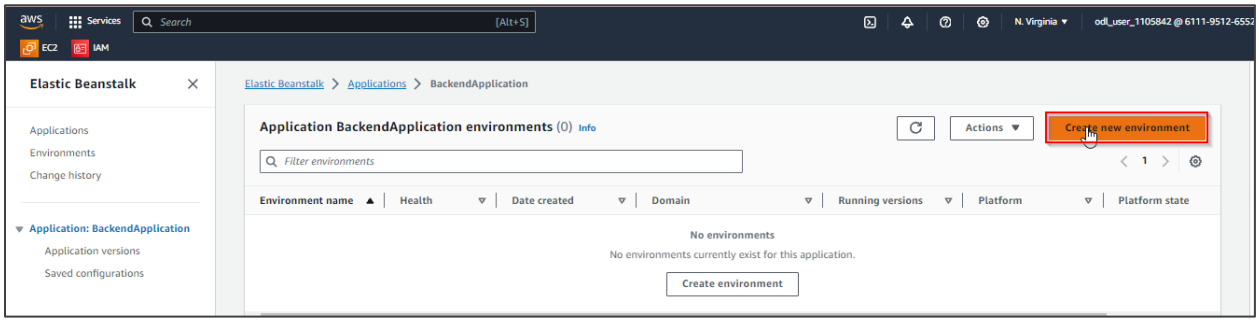


1.9 Enter a name and description for the application, and click on **Create**

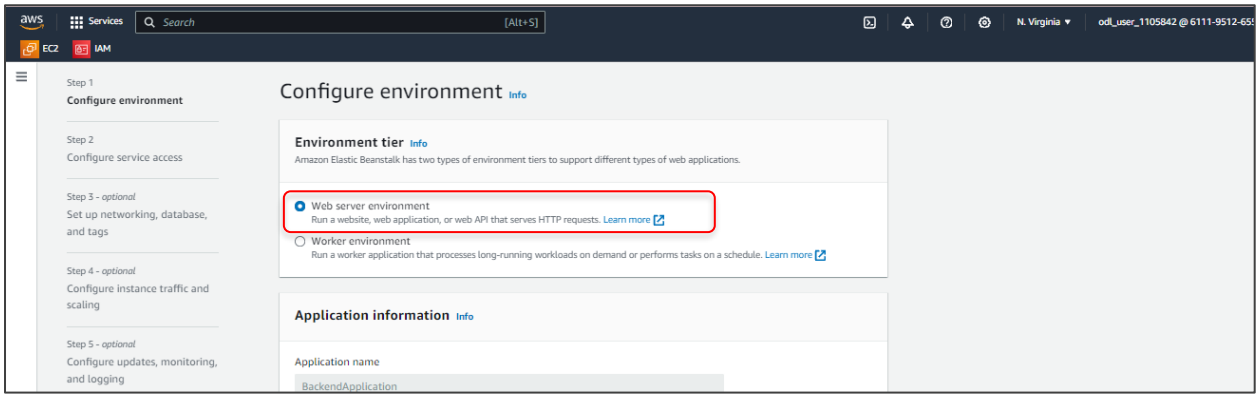


Step 2: Set up a Node.js Beanstalk environment for the backend application

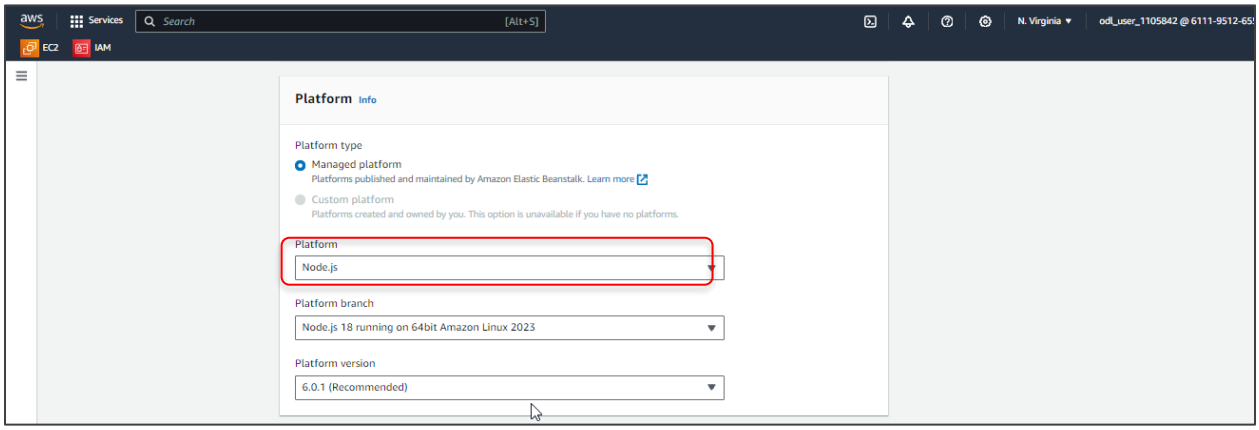
2.1 Click on **Create new environment** to set up the backend environment



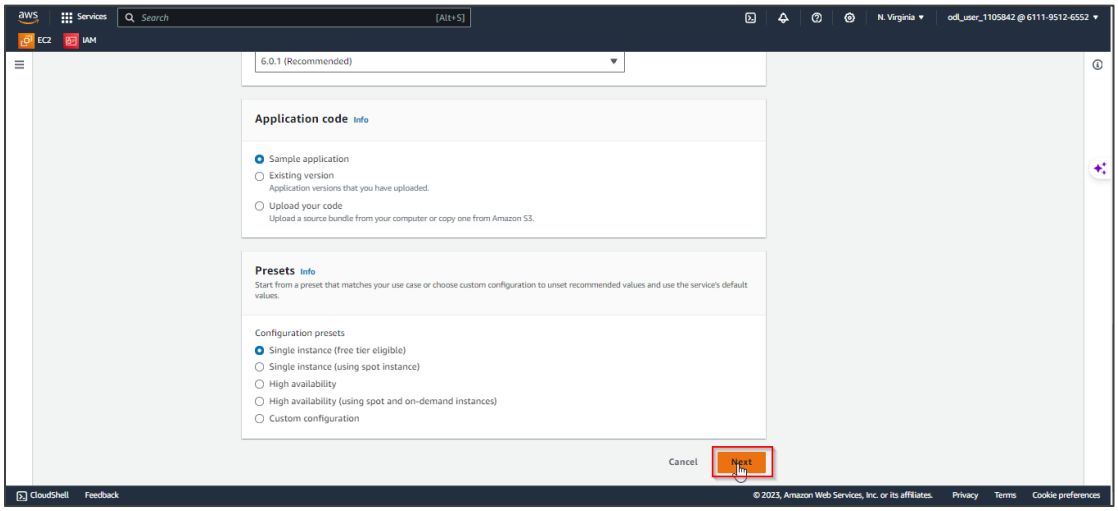
2.2 Choose the **Web server environment** as the environment tier



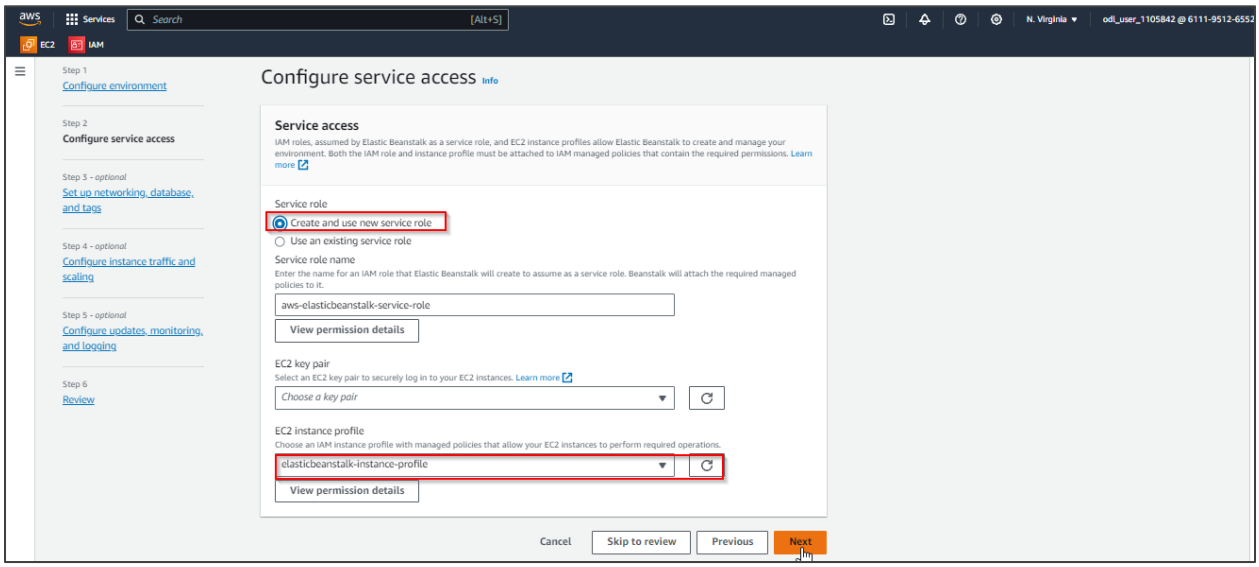
2.3 Choose **Node.js** as the environment platform



2.4 Click on **Next** while keeping all the default configurations

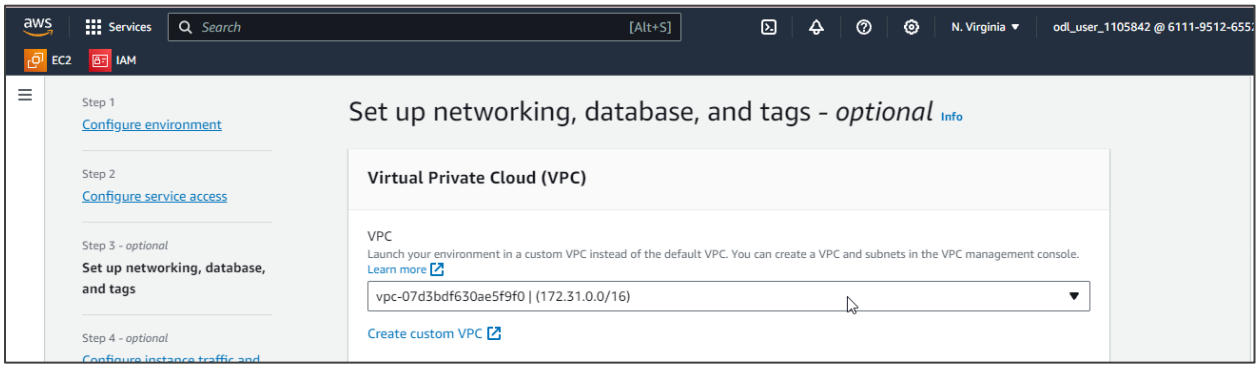


2.5 Choose the option to **Create and use new service role**, select the IAM instance profile, and then click on **Next**

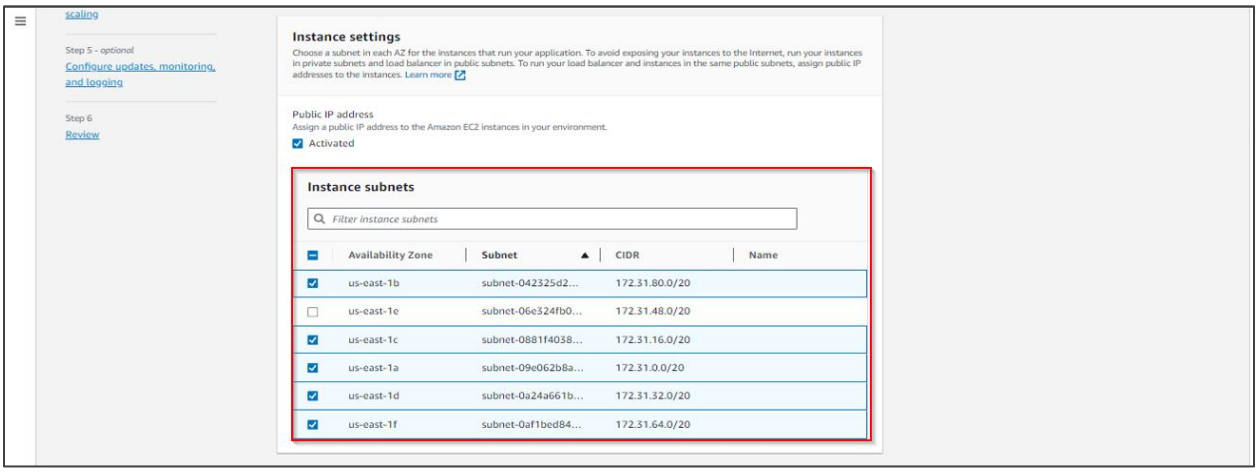


Note: Choose the IAM role which is already created in step 1.6.

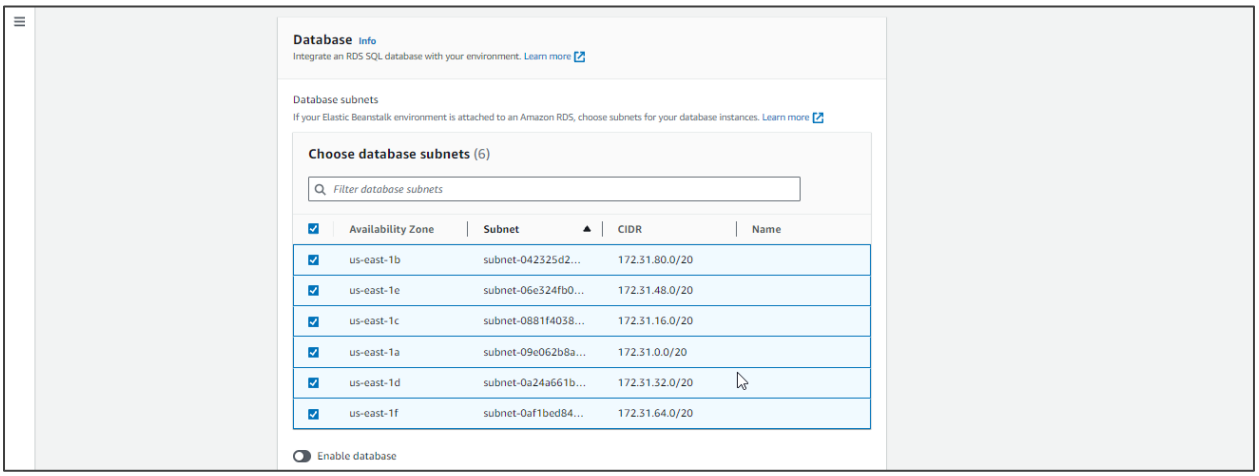
2.6 Choose the default **VPC**

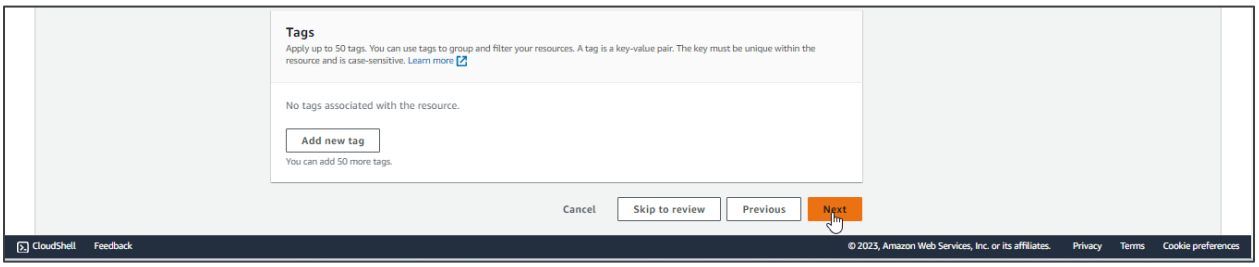


2.7 Under **Instance settings**, activate the **Public IP address,** and choose all the required **Instance subnets**

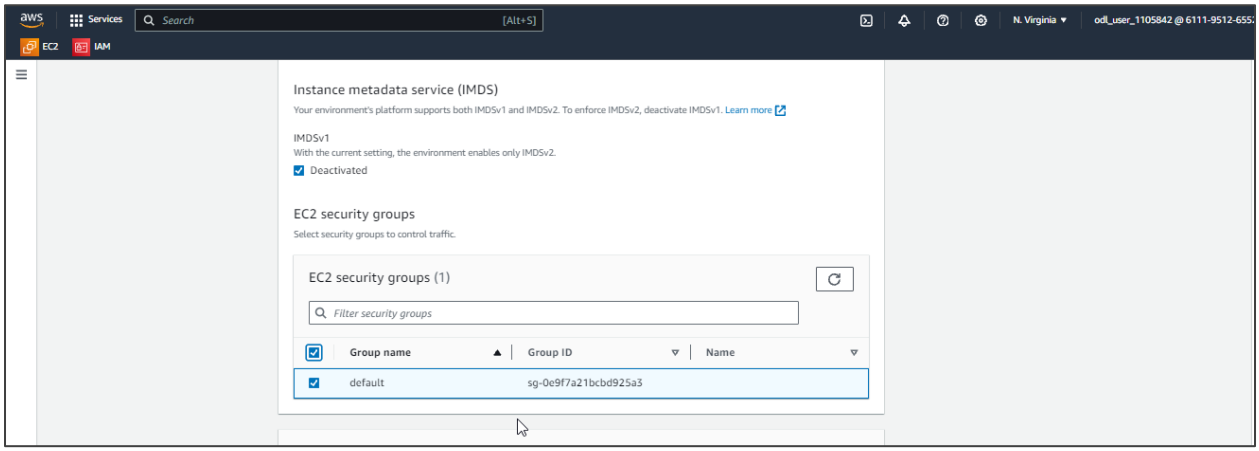


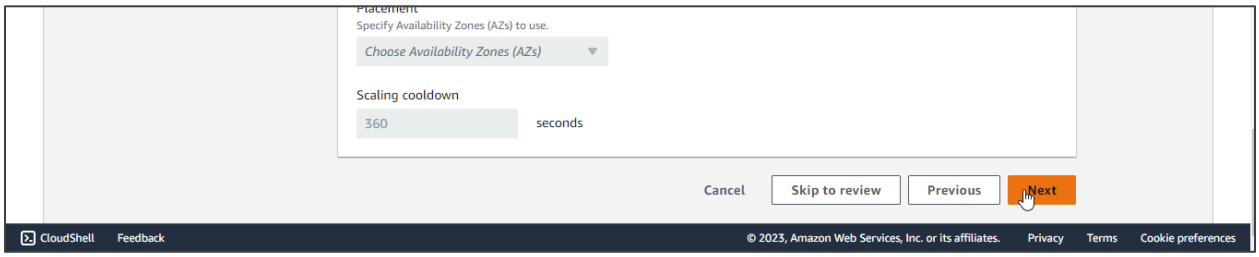
2.8 Choose all the required **database subnets** and click on **Next**



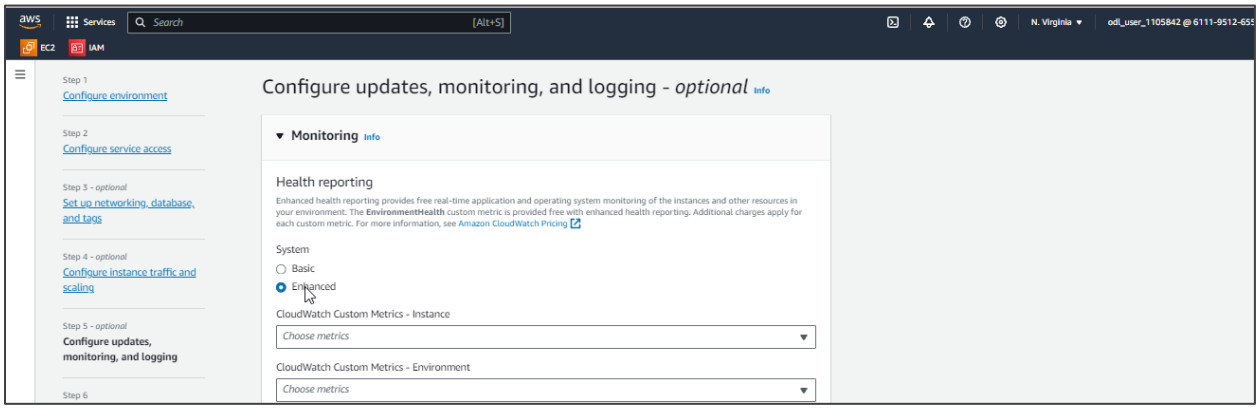


2.9 On the next page, select the default EC2 security group and click on **Next**

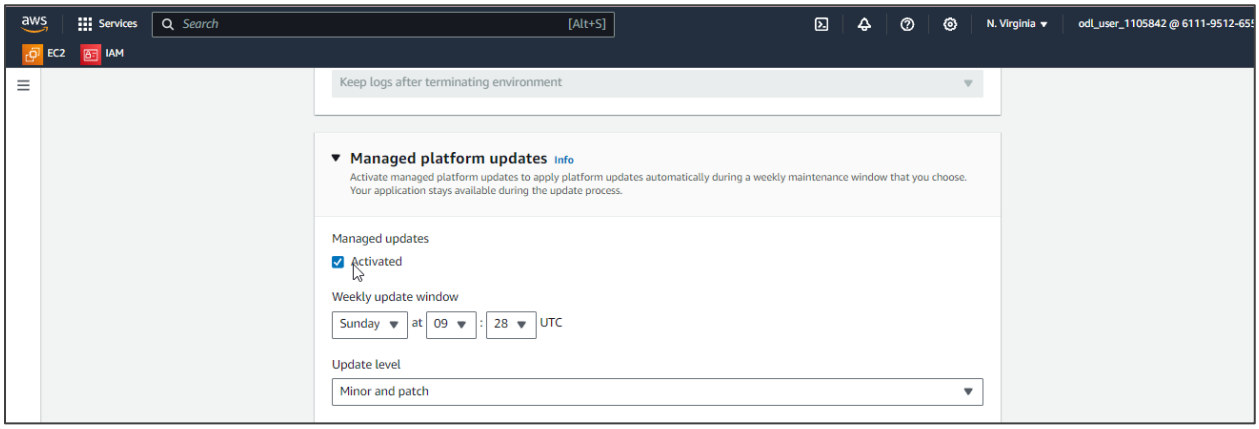




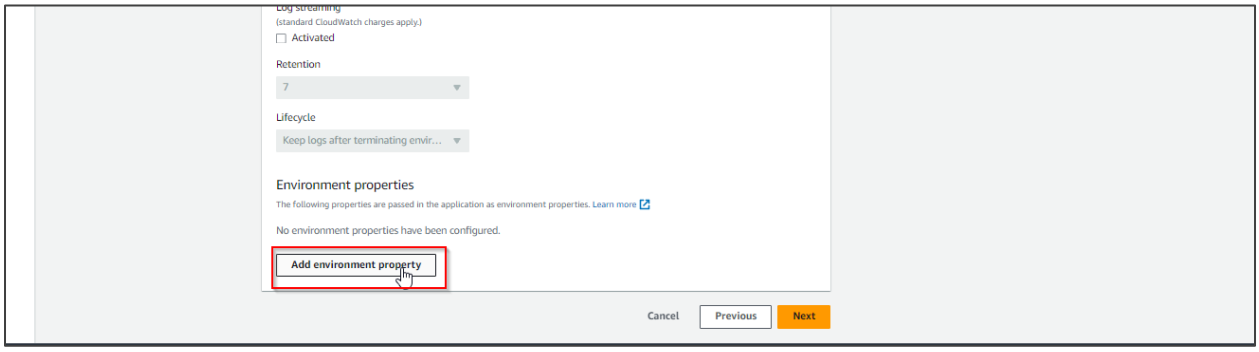
2.10 In the **Configure updates, monitoring, and logging** page, choose the enhanced health monitoring option for the Beanstalk environment



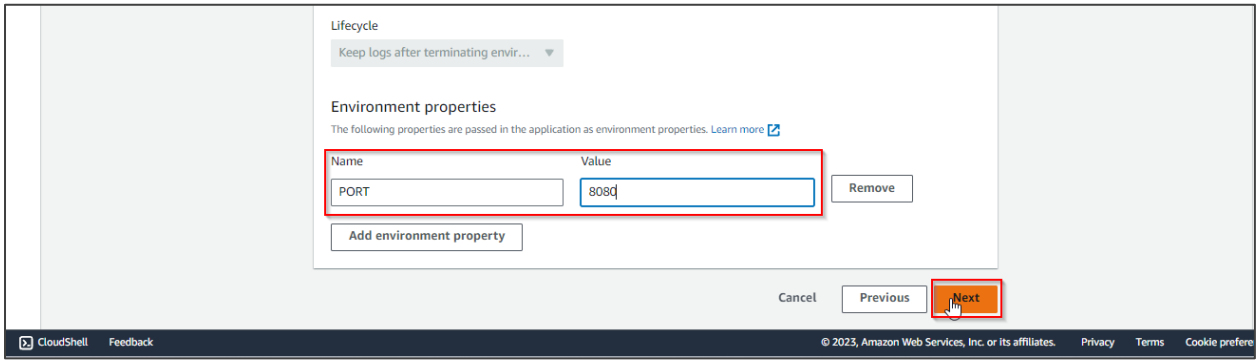
2.11 Activate the **Managed updates** to update the platform automatically



2.12 In the **Environment properties** section, click on **Add environment property** to specify the application environment property

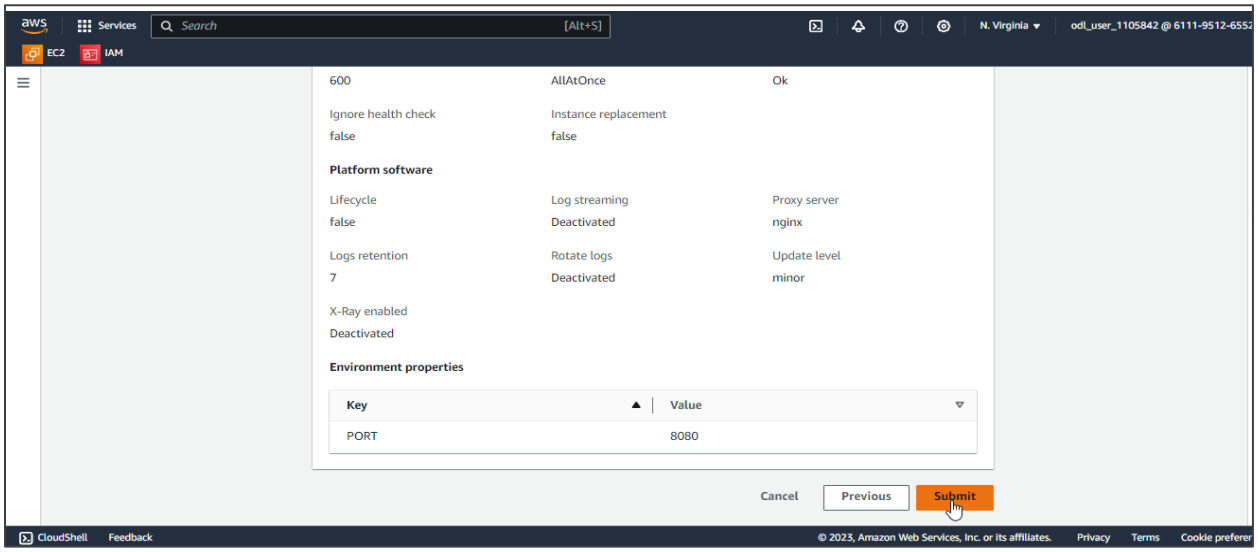


2.13 Provide **PORT** as an environment variable name and **8080** as its value, and then click on **Next**

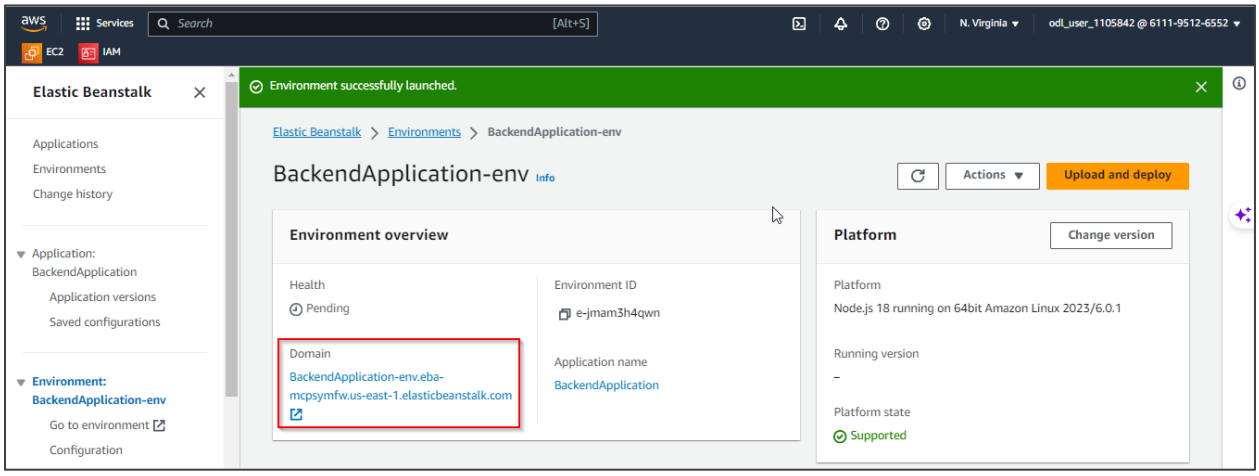


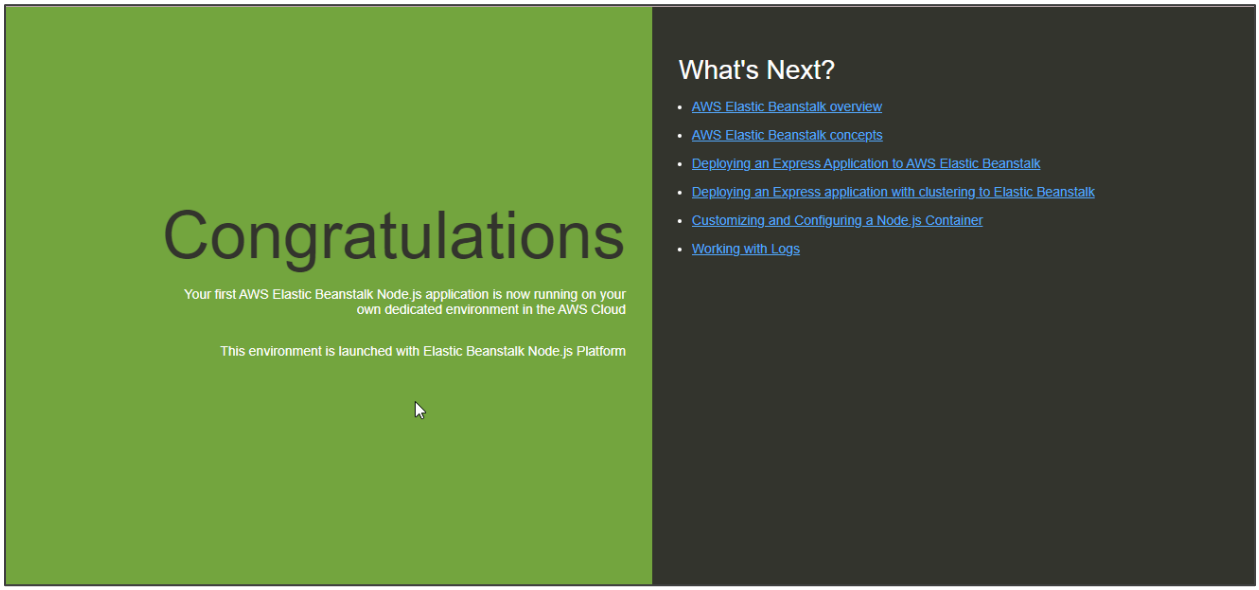
This environment property will ensure that the Node.js application is available on port 8080.

2.14 Review all the environment configurations and then click on **Submit**



2.15 Click the **Domain URL** to access the deployed sample application on Elastic Beanstalk

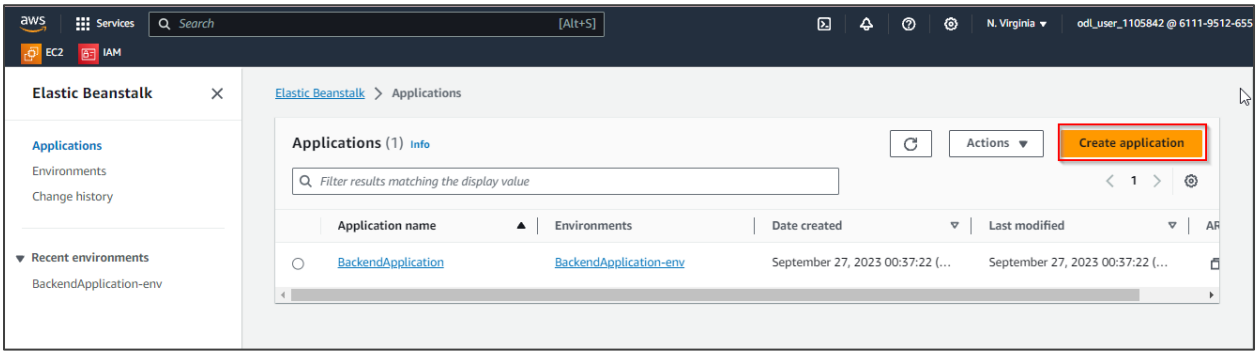




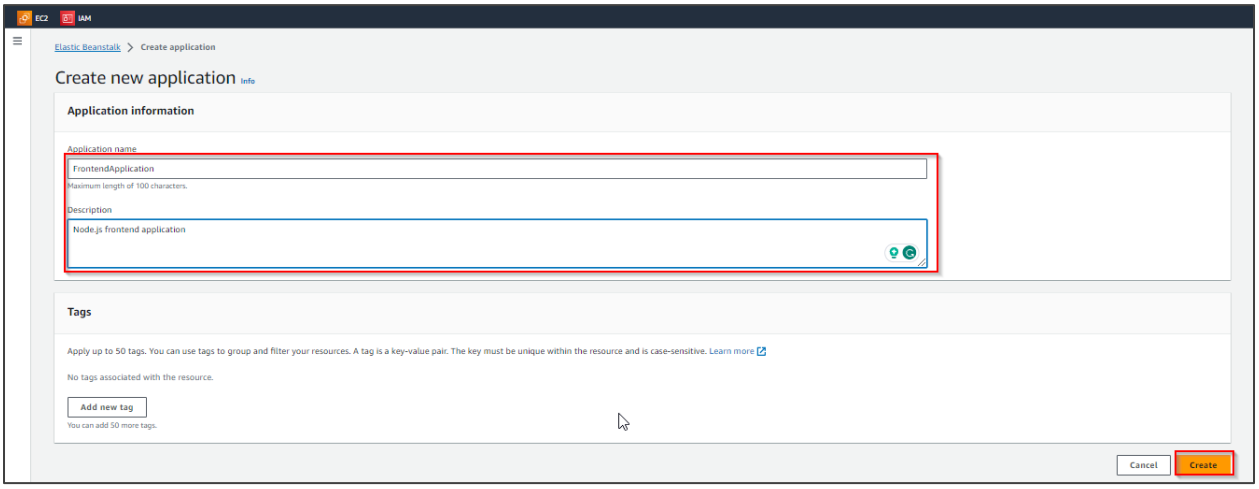
The sample application is deployed on the Elastic Beanstalk webserver.

Step 3: Set up an Elastic Beanstalk frontend application

3.1 Navigate back to the **Applications** page and click on **Create application** to create the frontend application



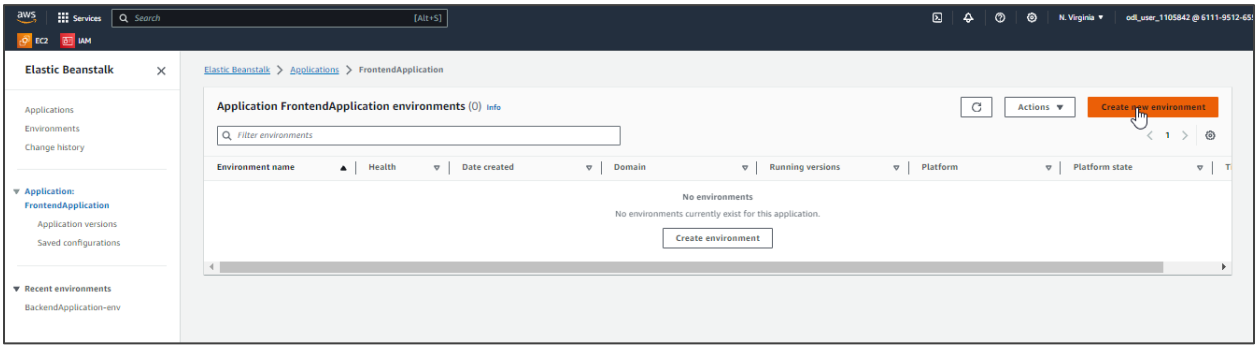
3.2 Enter a name and description for the application and click on **Create**



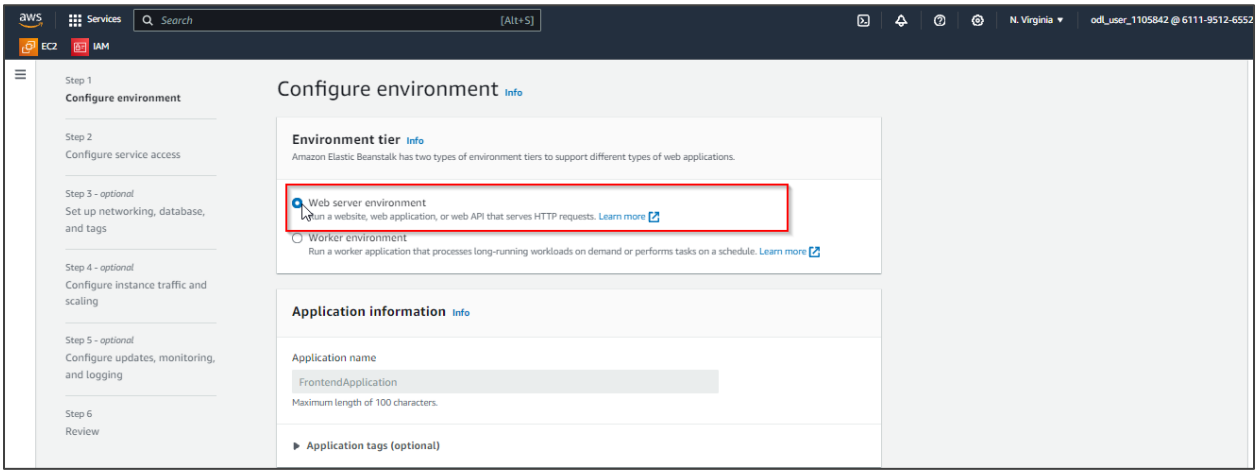
The Node.js application for the frontend will be created.

Step 4: Set up a Node.js Beanstalk environment for the frontend application

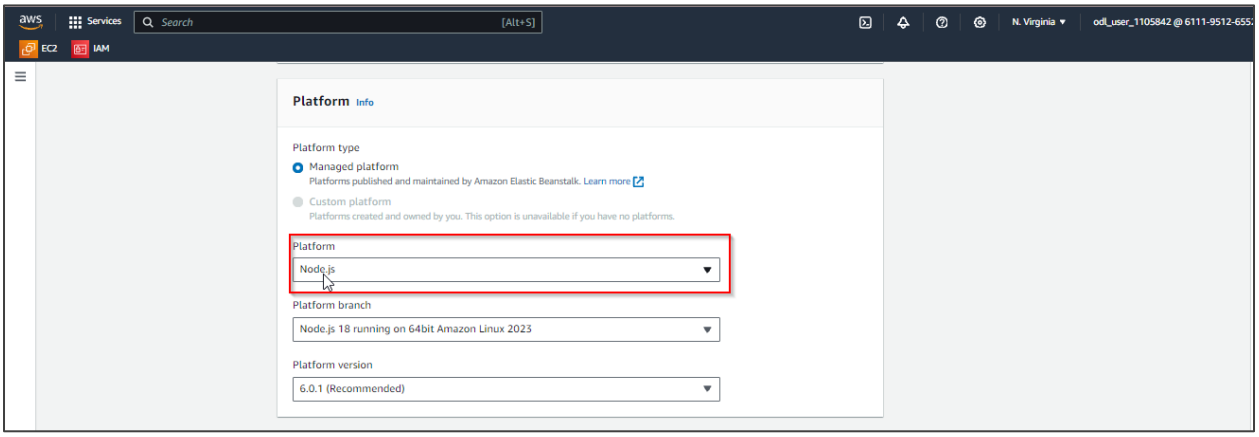
4.1 Click on **Create new environment** to set up the frontend application environment



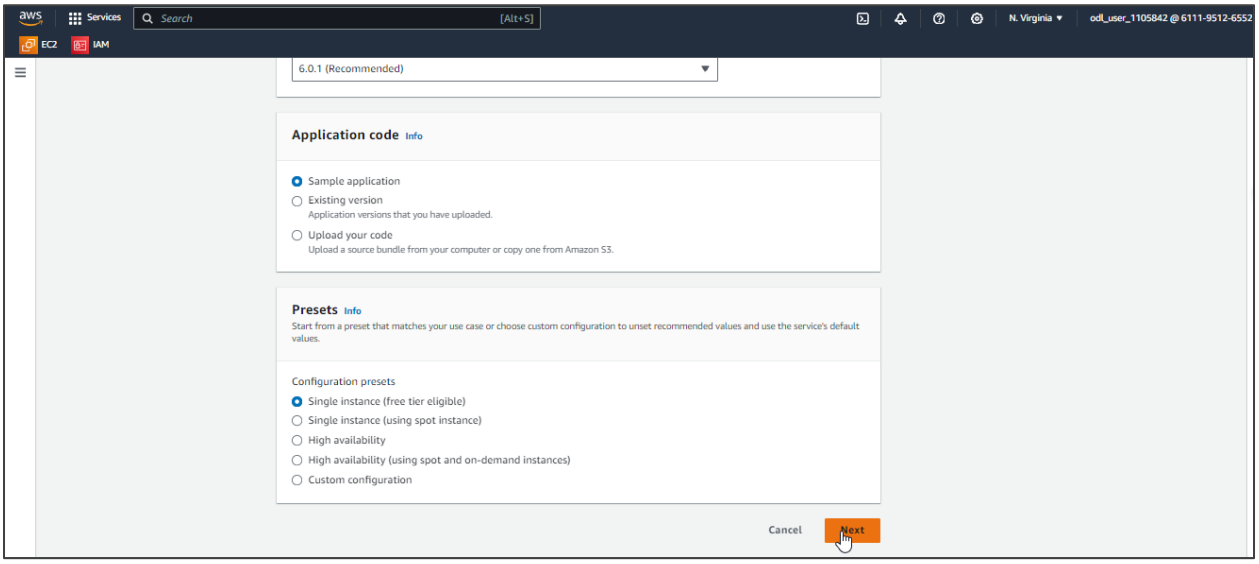
4.2 Choose the **Web server environment** as the environment tier



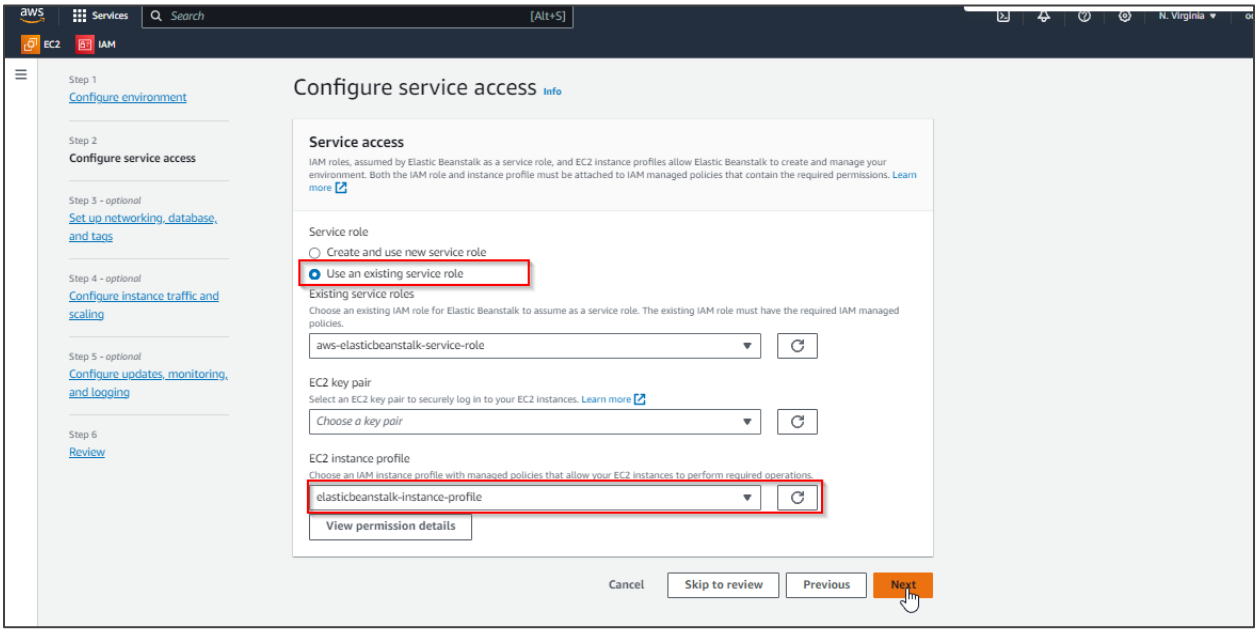
4.3 Select **Node.js** as the environment platform



4.4 Click on **Next** while keeping all the default configurations

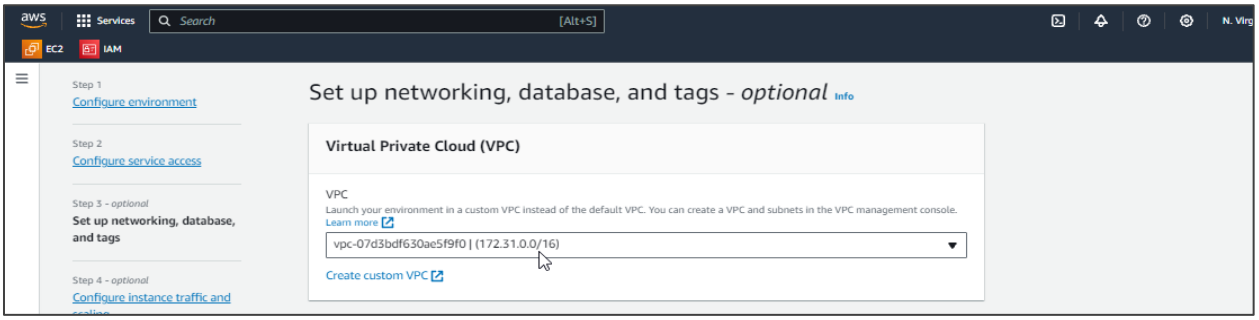


4.5 Choose the option **Use an existing service role**, select the IAM instance profile, and then click on **Next**

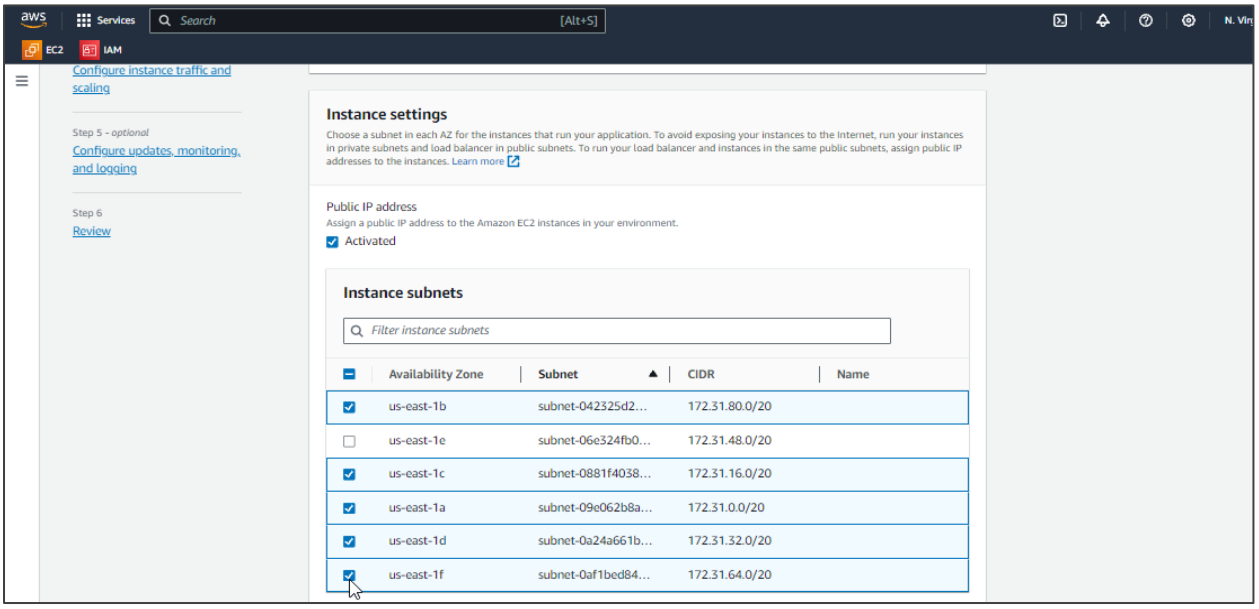


Note: Choose the IAM role which is already created in step 1.6

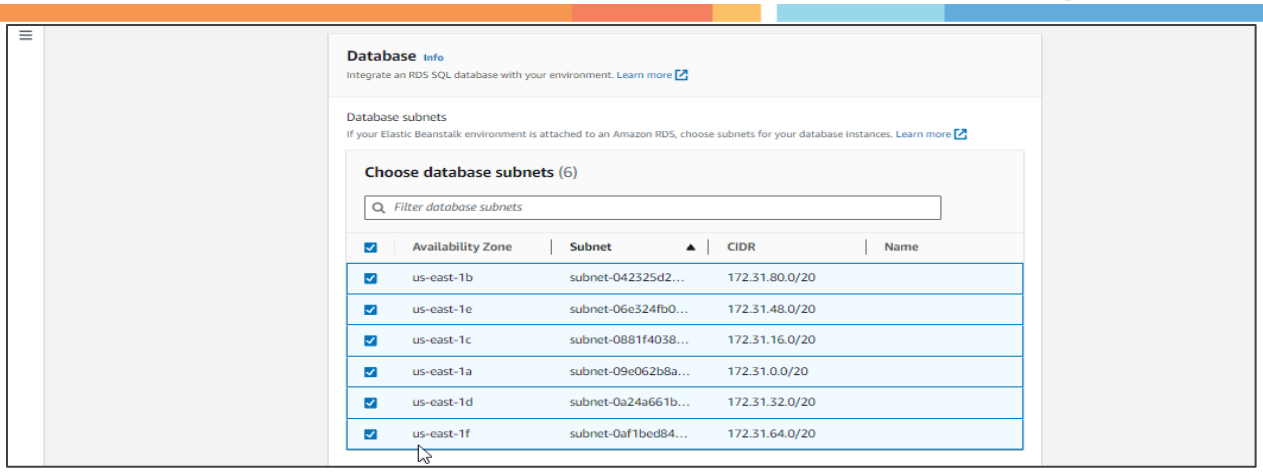
4.6 Select the default VPC

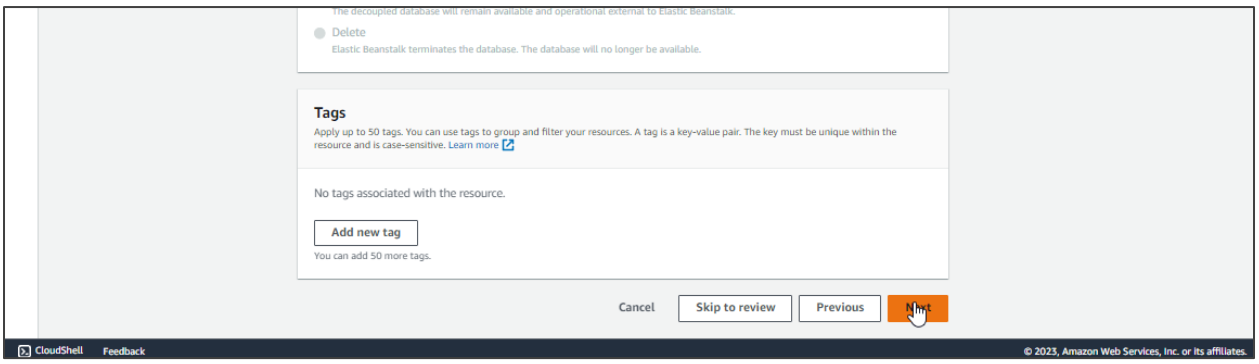


4.7 Activate the public IP address and choose all the required instance subnets

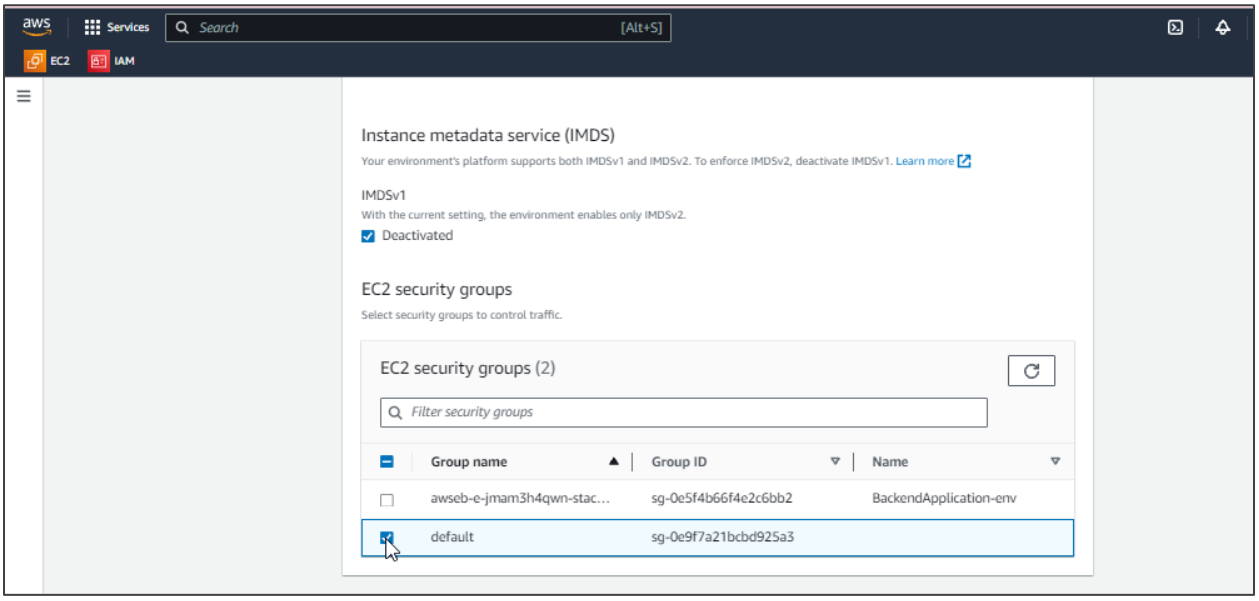


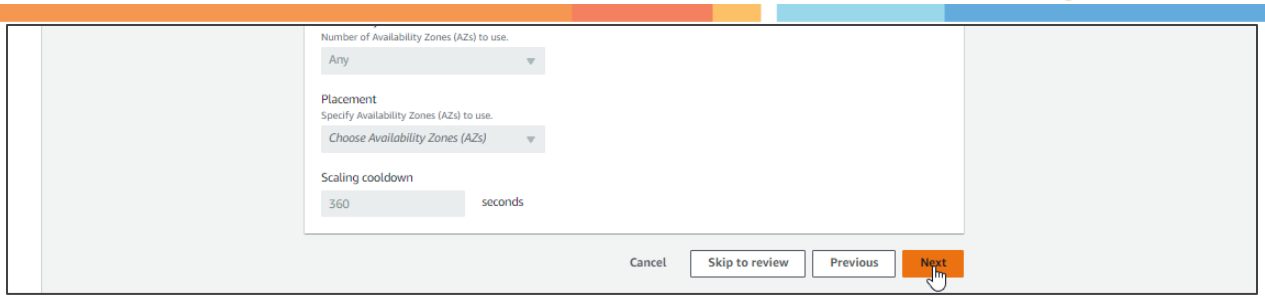
4.8 Select all the required database subnets and click on **Next**



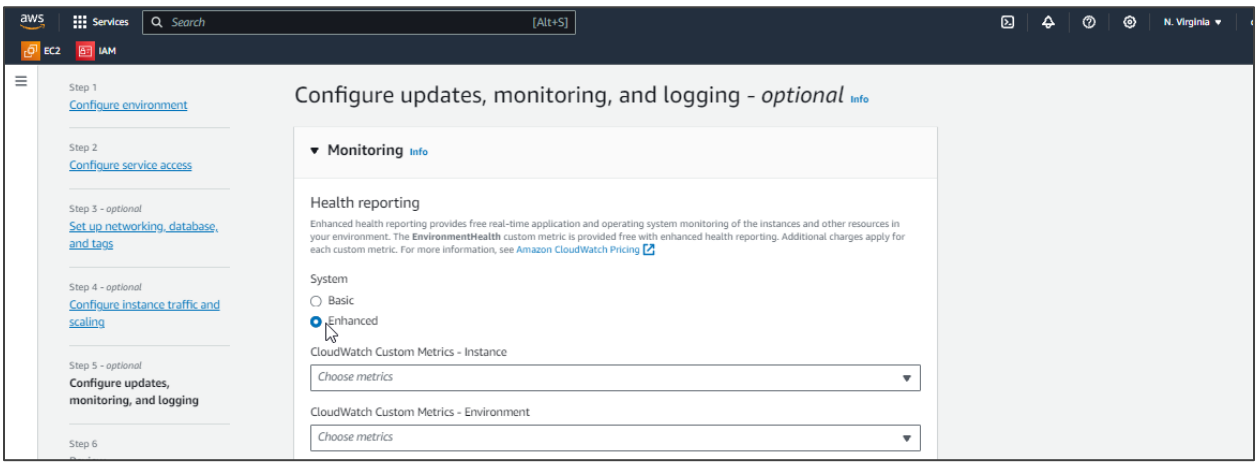


4.9 On the next page, select the default EC2 security group and click on **Next**

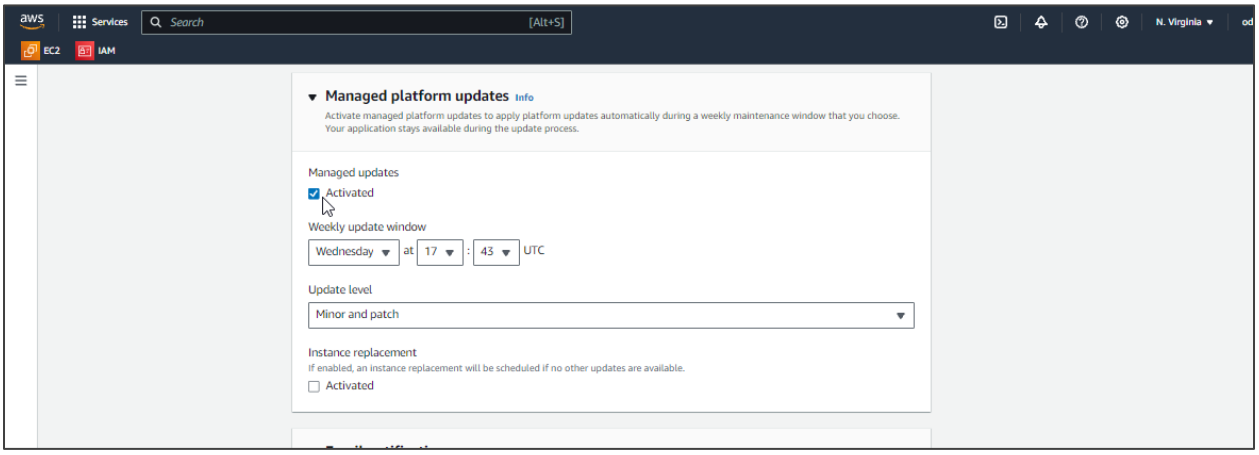




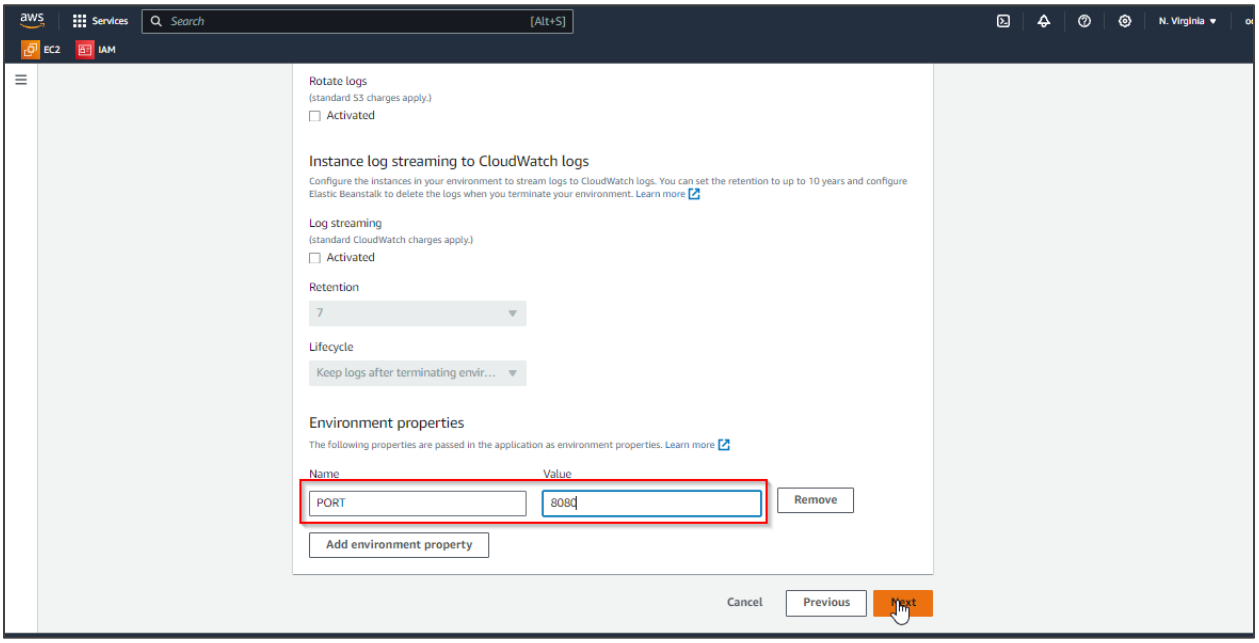
4.10 In the **Configure updates, monitoring, and logging** page, select the Enhanced health reporting option for the Beanstalk environment



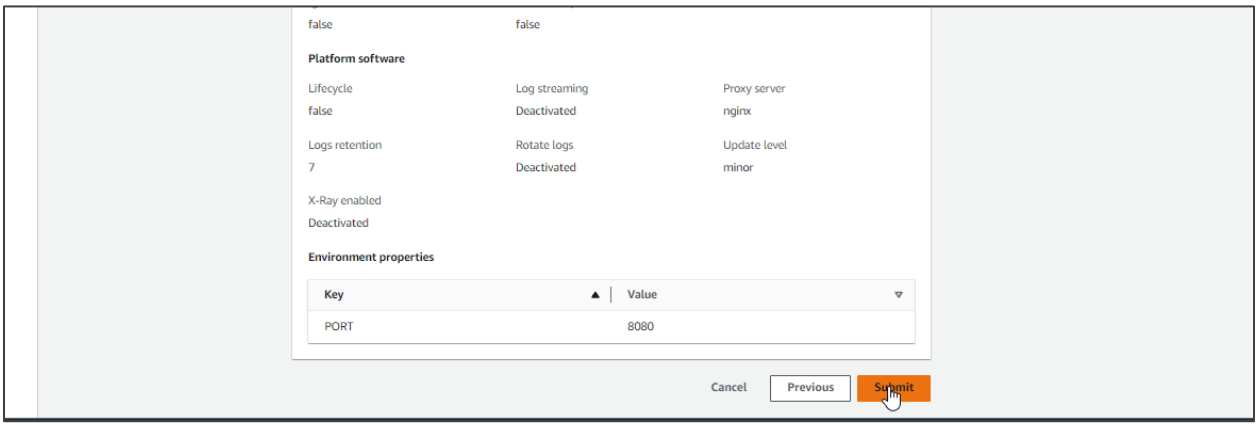
4.11 Activate the **Managed updates** to update the platform automatically



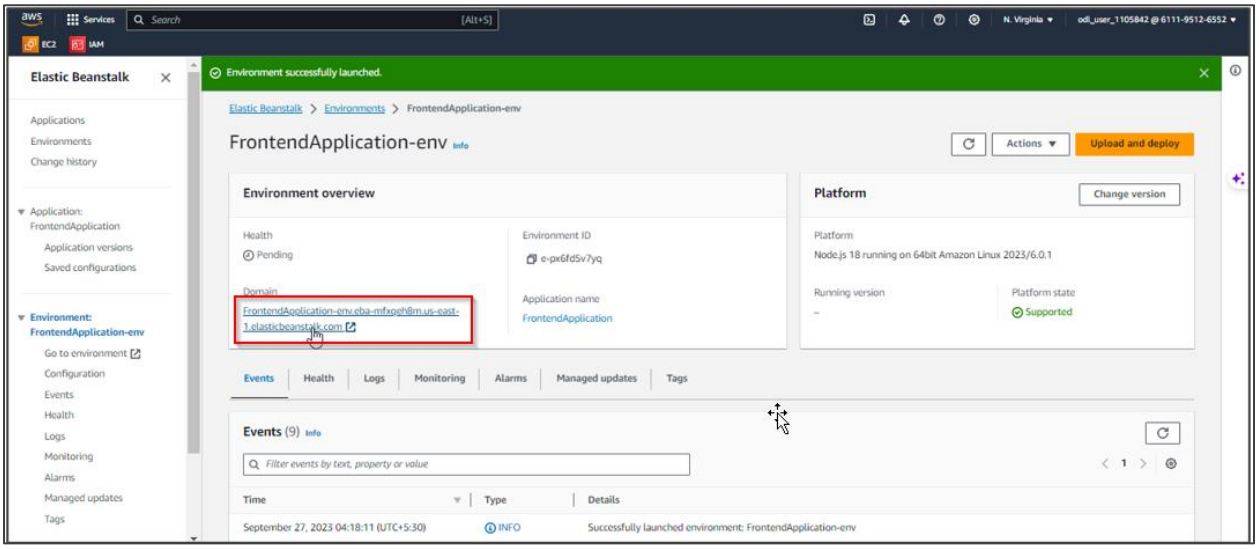
4.12 Provide **PORT** as an environment variable using which the Node.js application will be available on port **8080,** and then click on **Next**

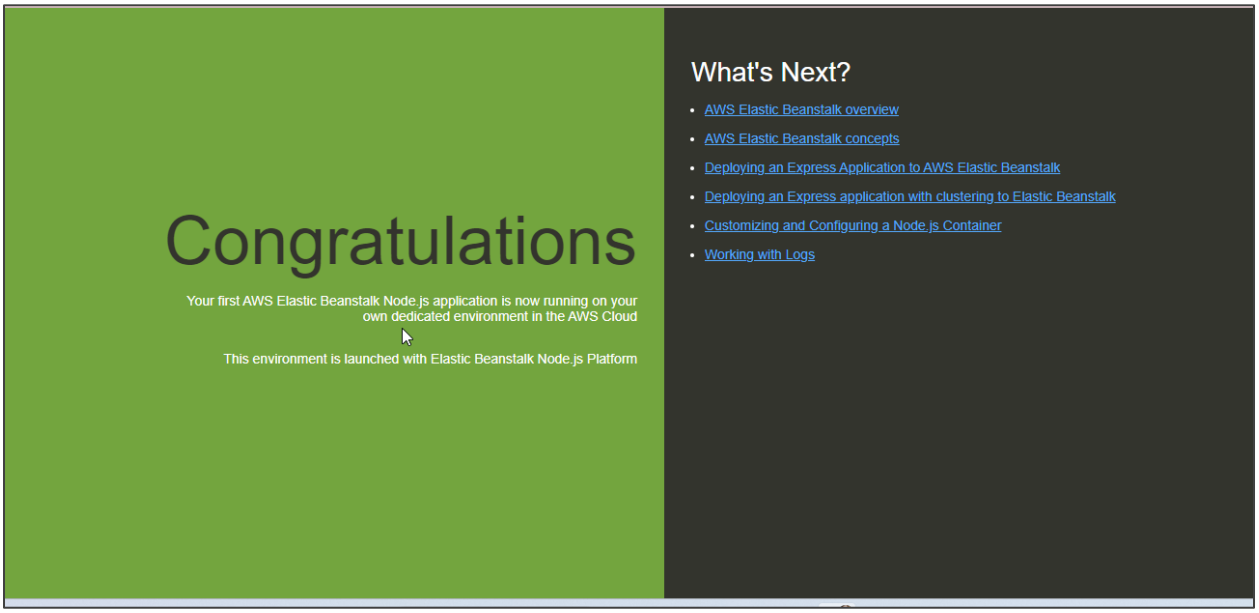


4.13 Review all the environment configurations and then click on **Submit**



4.14 Click the **Domain URL** to access the sample application, which is deployed on the Elastic Beanstalk





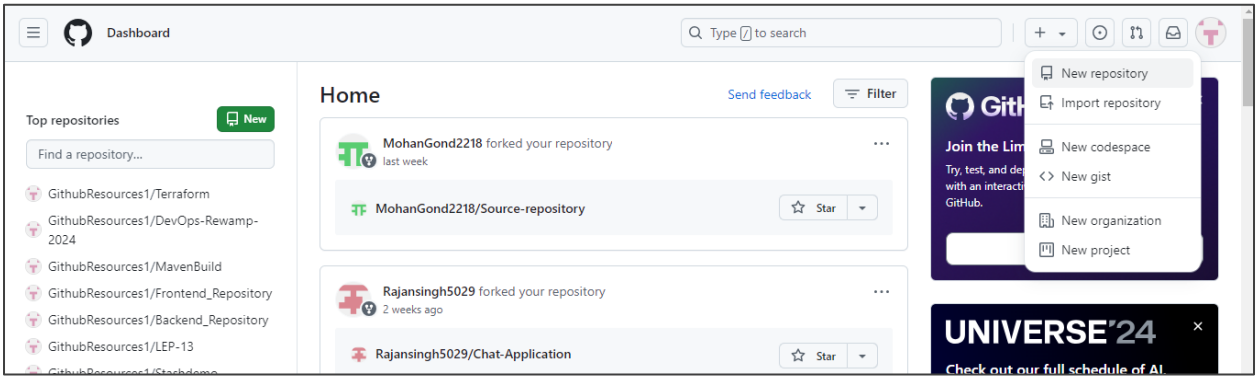
The sample application is deployed on the Elastic Beanstalk webserver.

Step 5: Set up the CodePipeline for the backend application

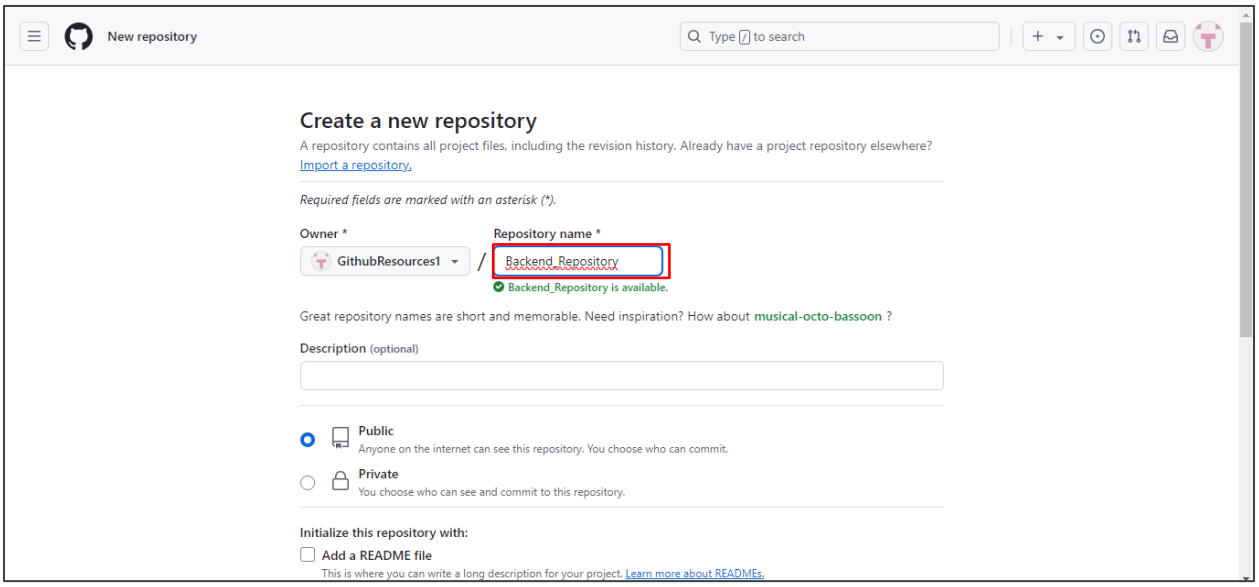
Note: Create two separate GitHub repositories for the backend and frontend by importing repositories from GitHub using the following HTTPS clone URLs:

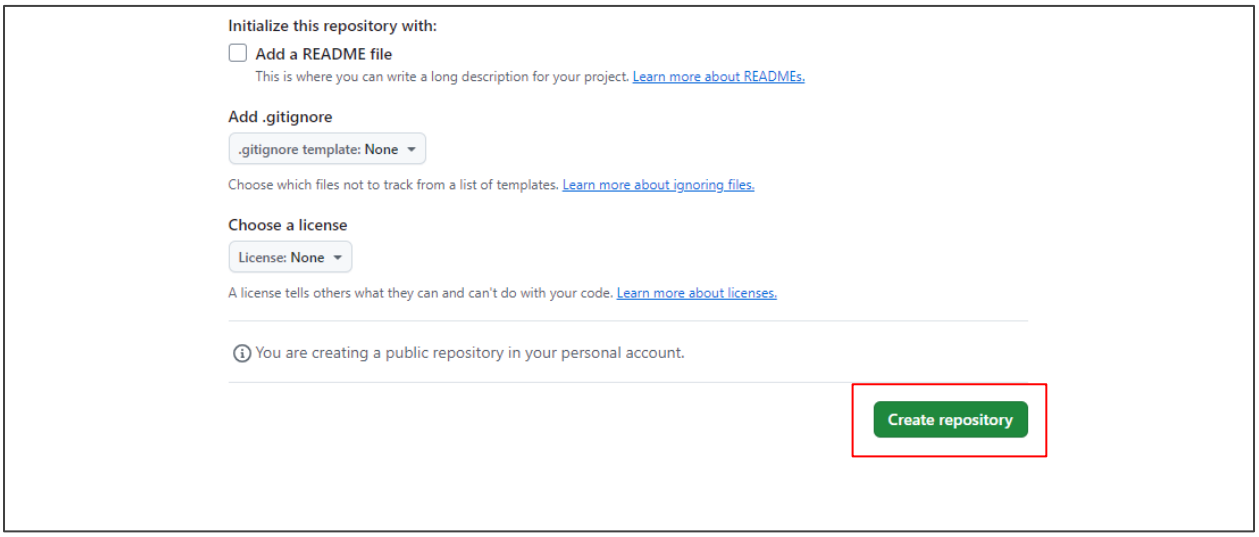
https://github.com/anujdevopslearn/NodeBackend.git for **Backend_Repository**
https://github.com/anujdevopslearn/ReactFrontend.git for **Frontend_Repository**

5.1 Navigate to the **+** icon and click on **New repository**

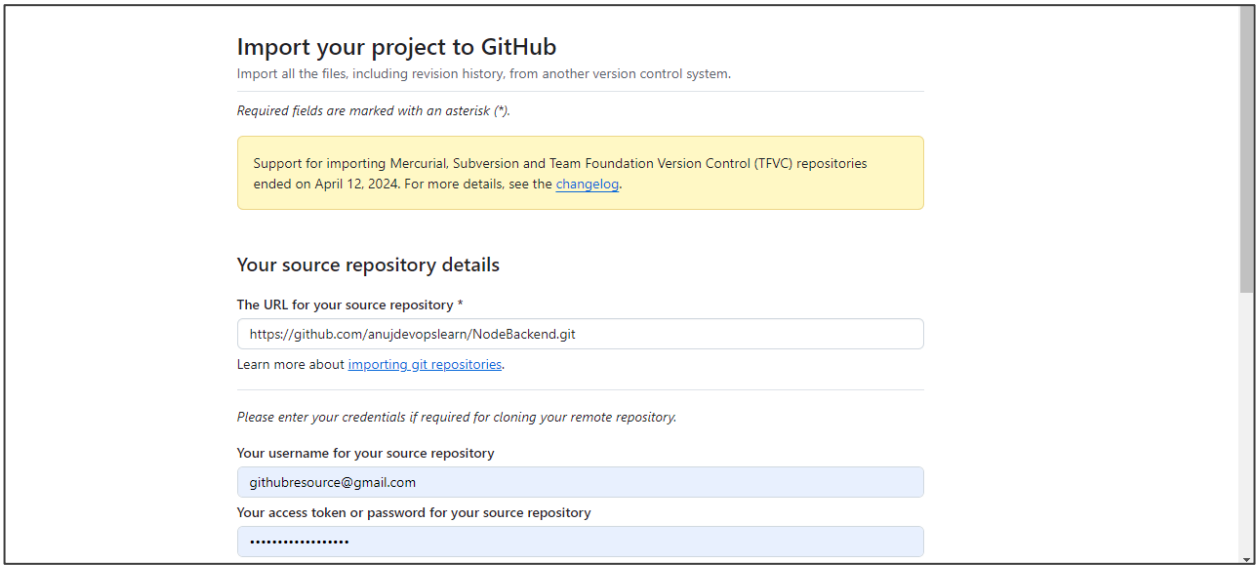


5.2 Add the **Repository name** as **Backend_Repository** and click on **Create repository**

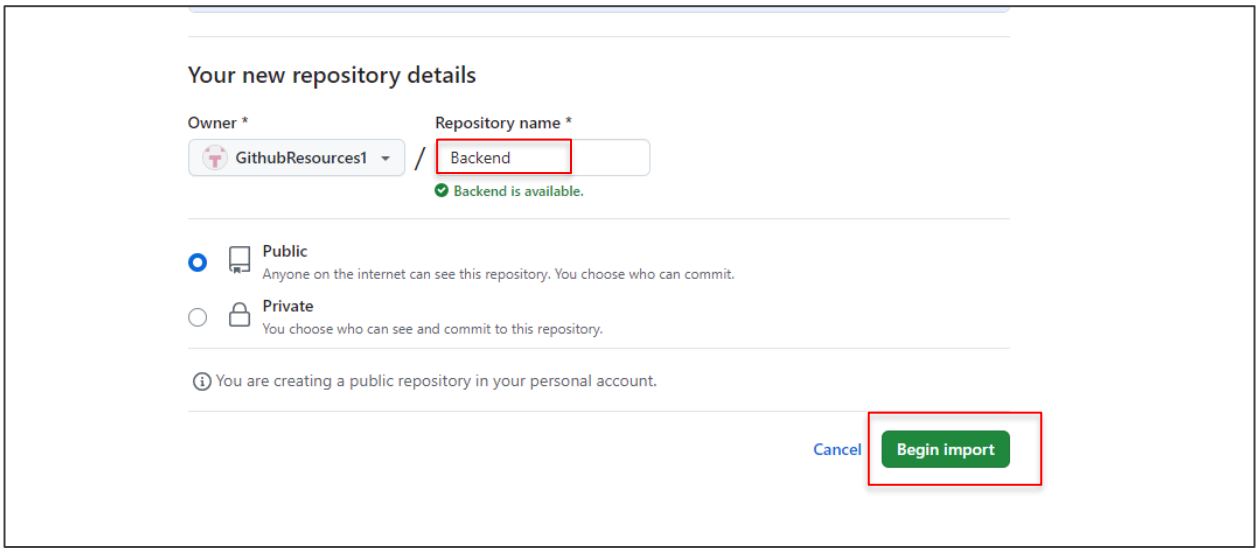




5.3 Now, navigate directly to the GitHub Importer using the URL below, then enter the repository URL you want to import. For the backend, use:
https://github.com/new/import
https://github.com/anujdevopslearn/NodeBackend.git

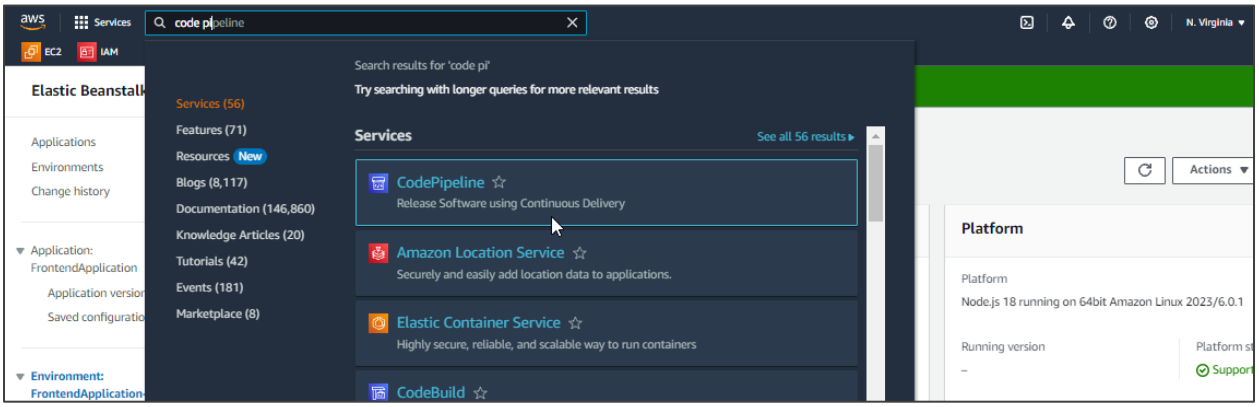


5.4 Add the Repository name as **Backend** and click on **Begin import**

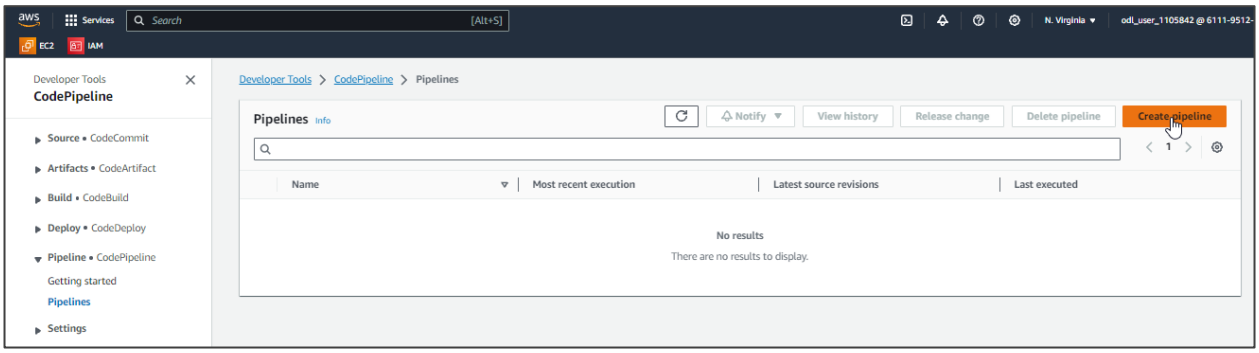


Note: Similarly create for Frontend_Repository.

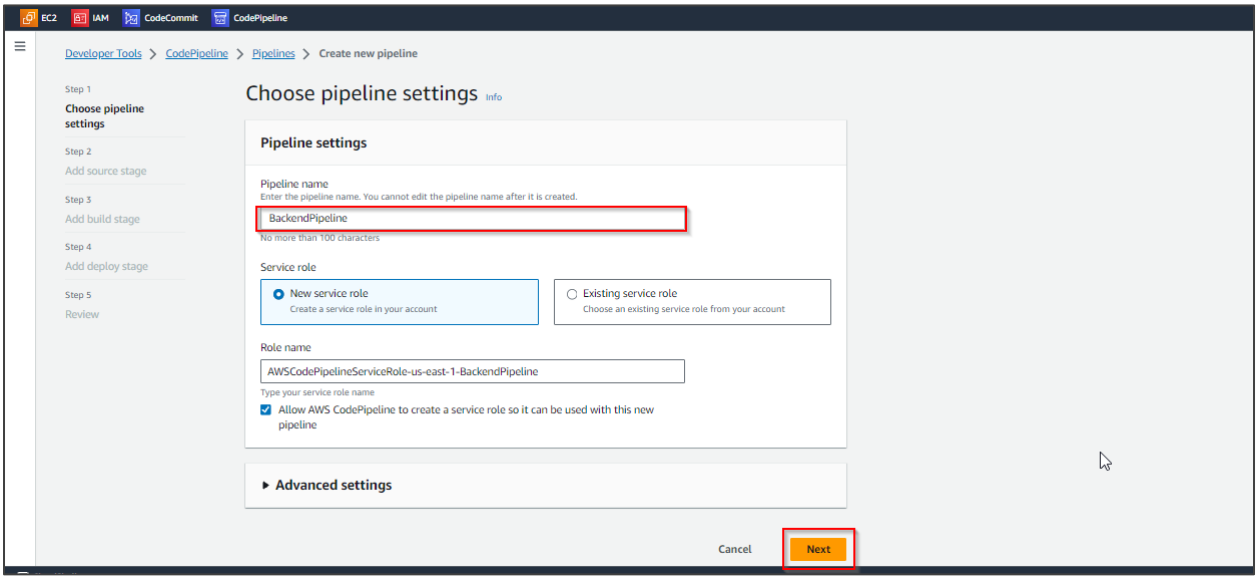
5.5 Navigate to the AWS CodePipeline service to create a new pipeline



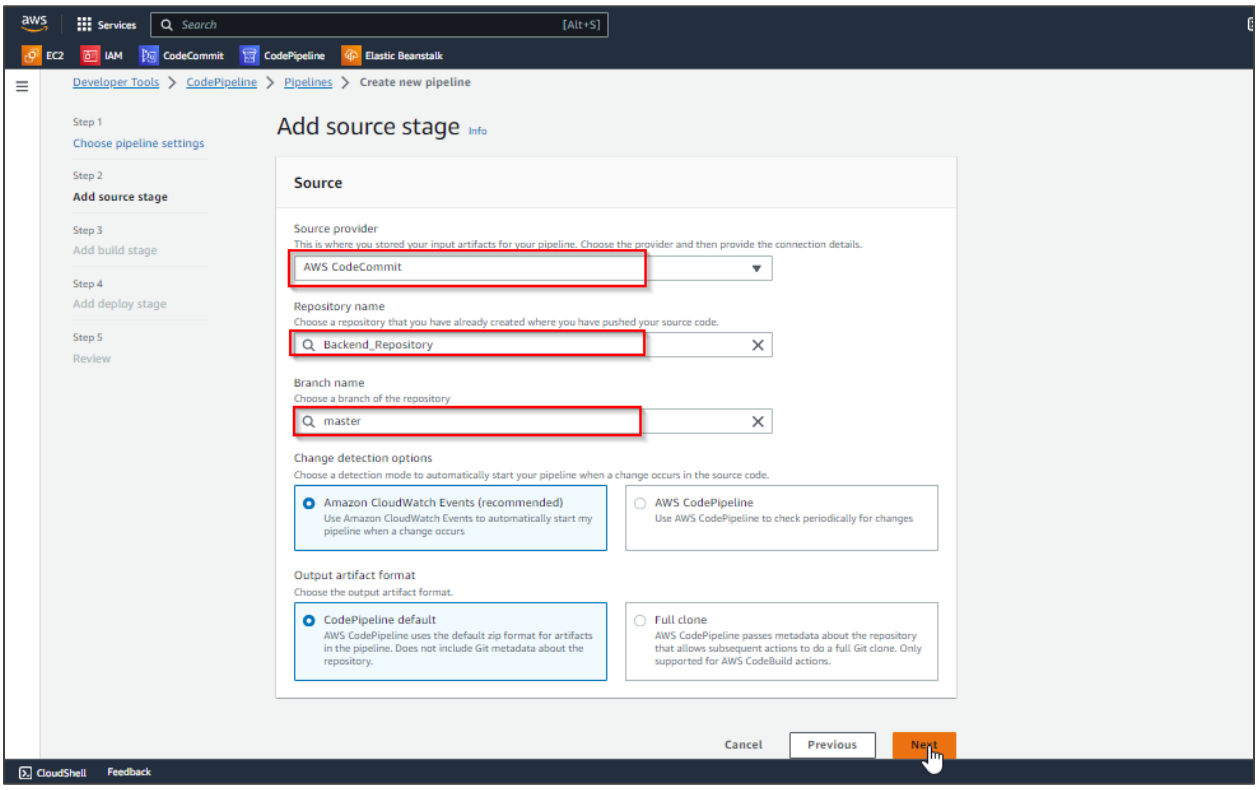
5.6 Click on **Create pipeline**



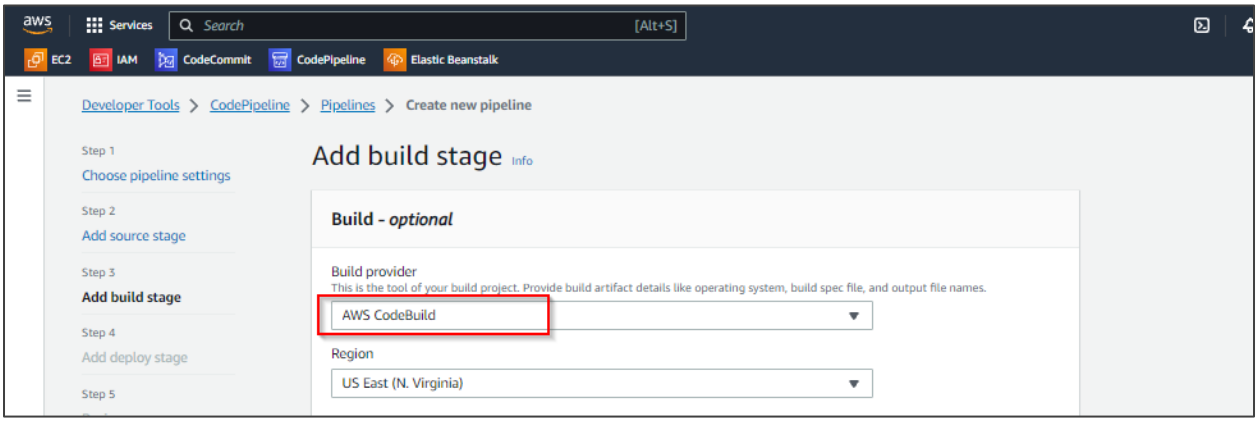
5.7 Enter a suitable name for the backend pipeline, retain all the default configurations, and click on **Next**



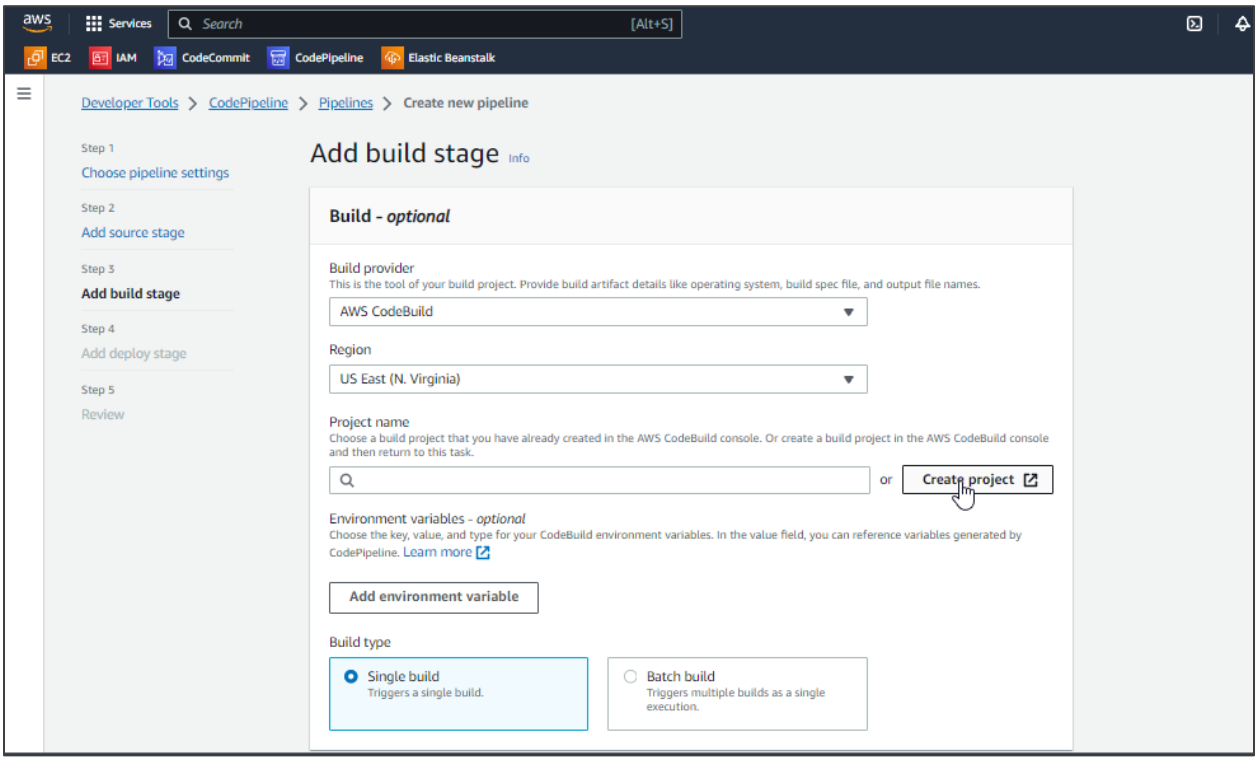
5.8 In the **Add source stage** page, select **AWS CodeCommit** as the Source provider. Specify the backend repository name and branch, select **Amazon CloudWatch Events** as the monitoring service, and click **Next**.



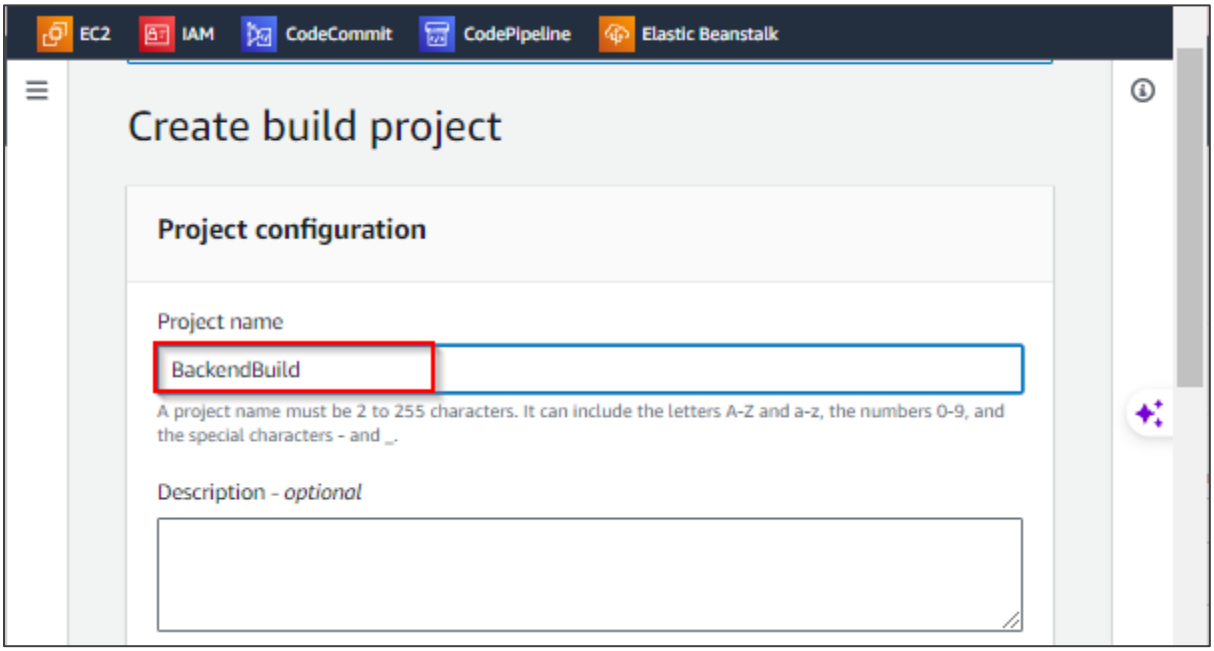
5.9 Select **AWS CodeBuild** as the Build provider



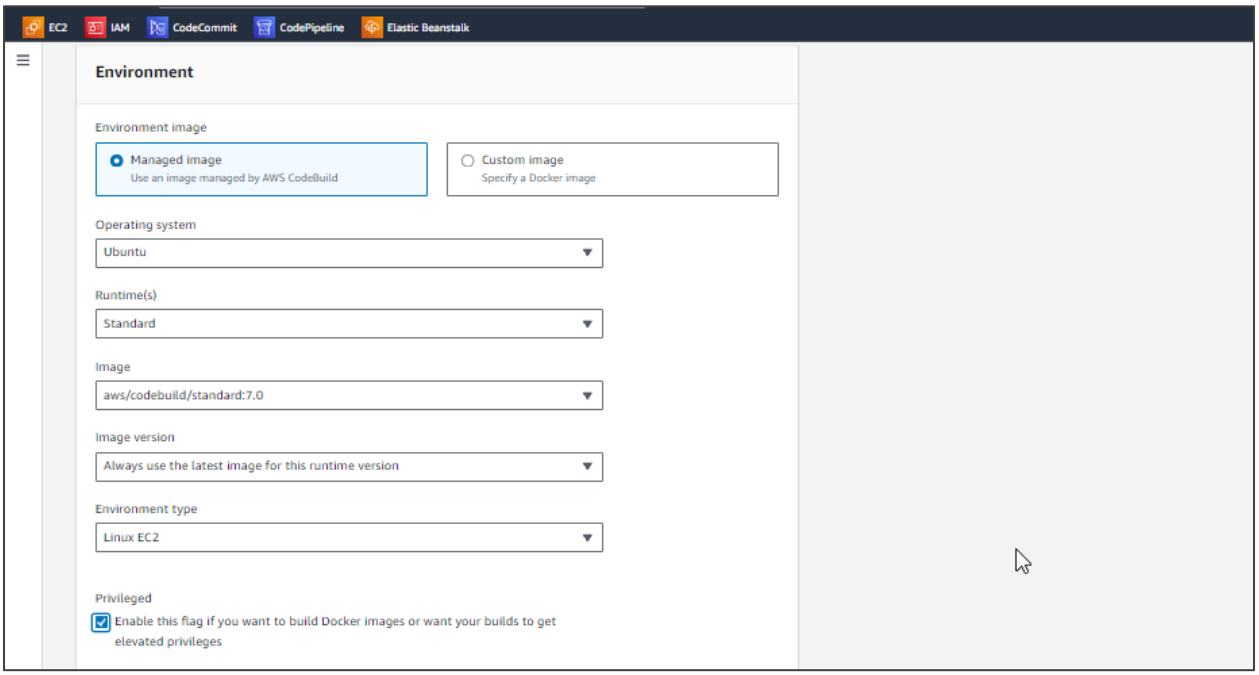
5.10 Click on **Create project** to create a new CodeBuild project



5.11 Enter a name for the CodeBuild project

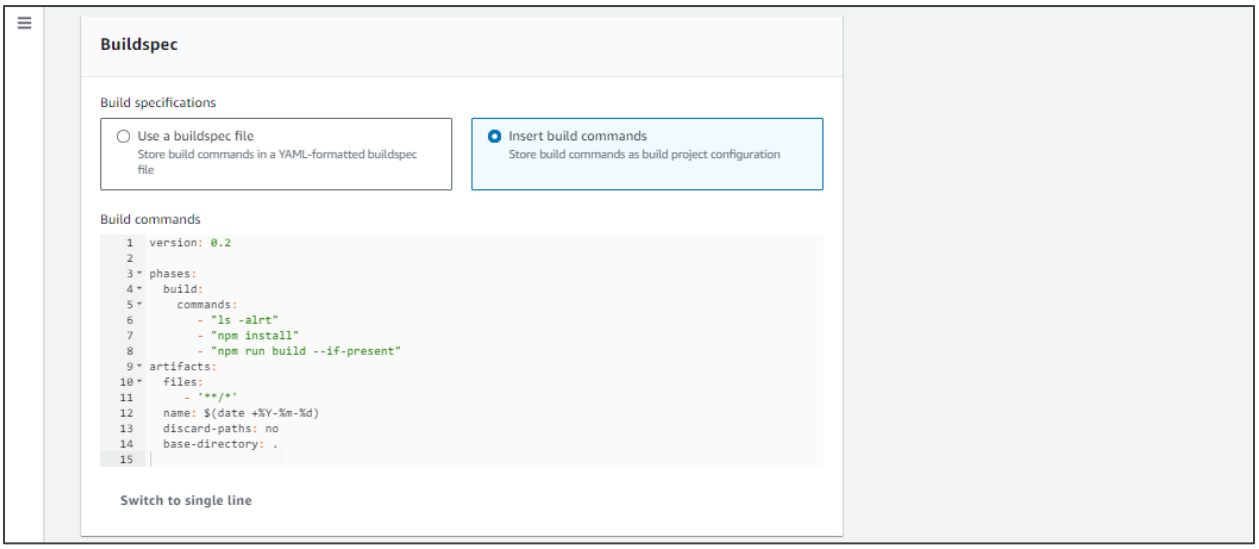


5.12 Under environment configurations, select **Ubuntu** as the Operating system, **Standard** as the Runtime(s), **aws/codebuild/standard:7.0** as Image, and **Linux EC2** as the environment type

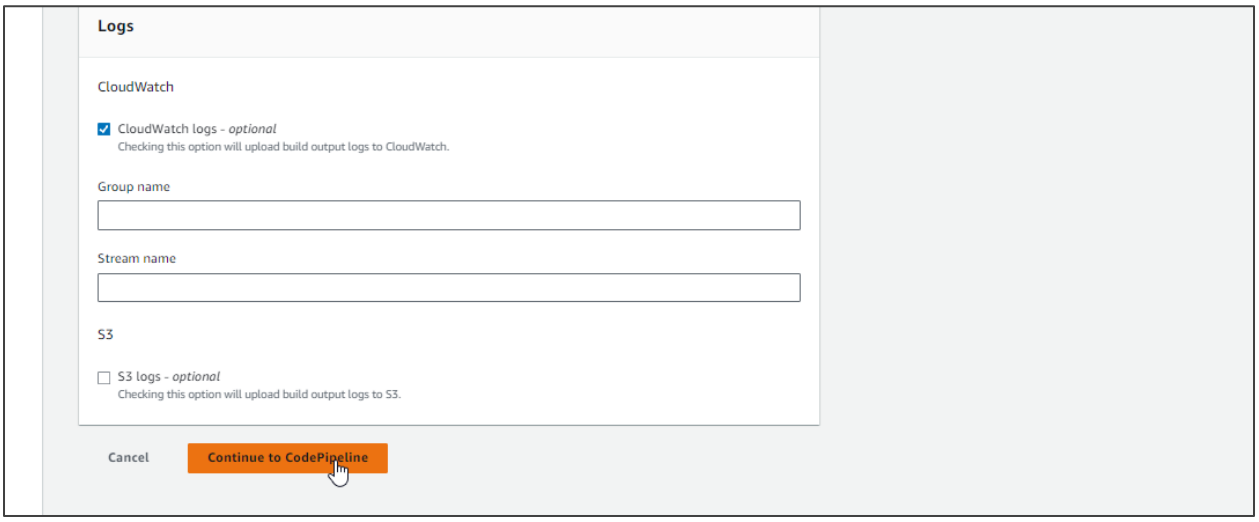


5.13 In the **Buildspec** configuration, insert the following YAML code:

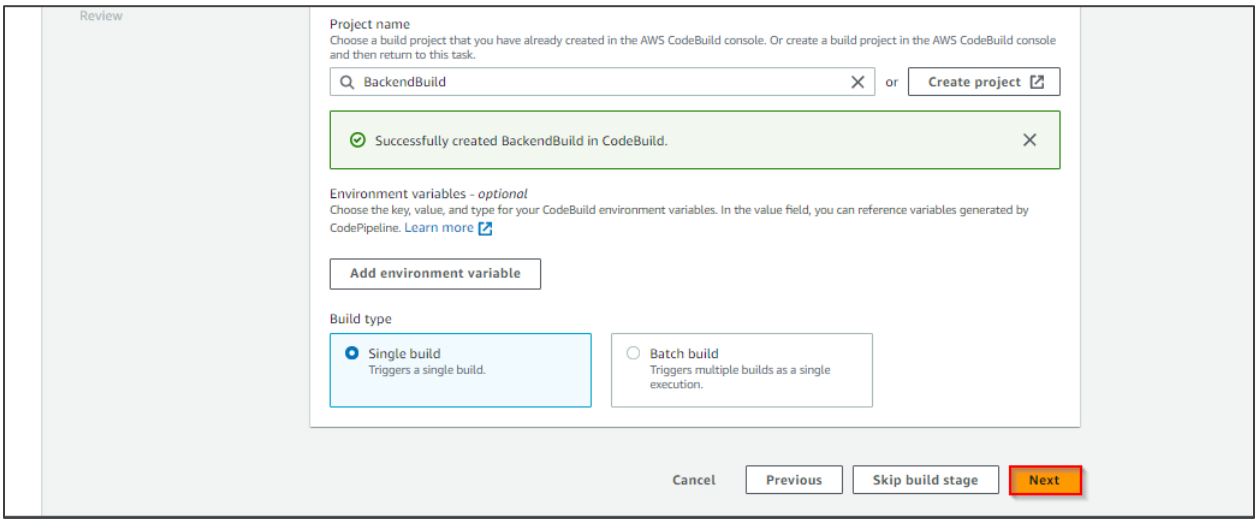
```
version: 0.2

phases:
  build:
    commands:
      - "ls -alrt"
      - "npm install"
      - "npm run build --if-present"
artifacts:
  files:
    - '**/*'
  name: $(date +%Y-%m-%d)
  discard-paths: no
  base-directory: .
```



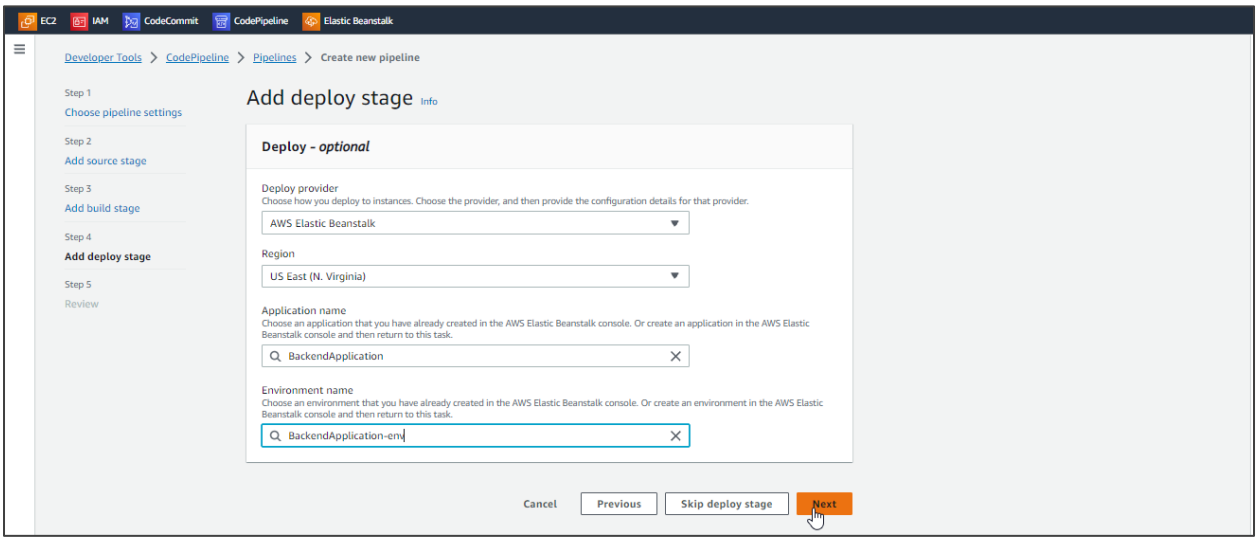
5.14 Click on **Continue to CodePipeline**



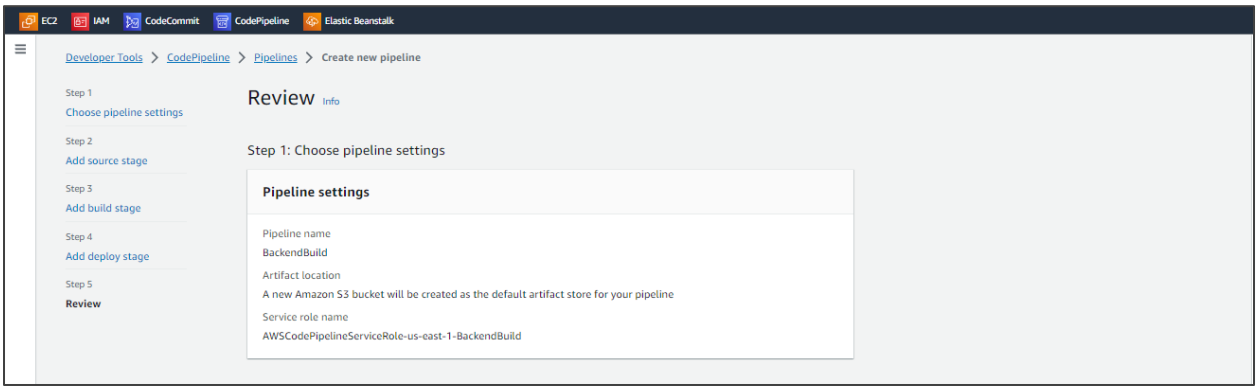
5.15 After successfully creating the CodeBuild project, click on **Next**

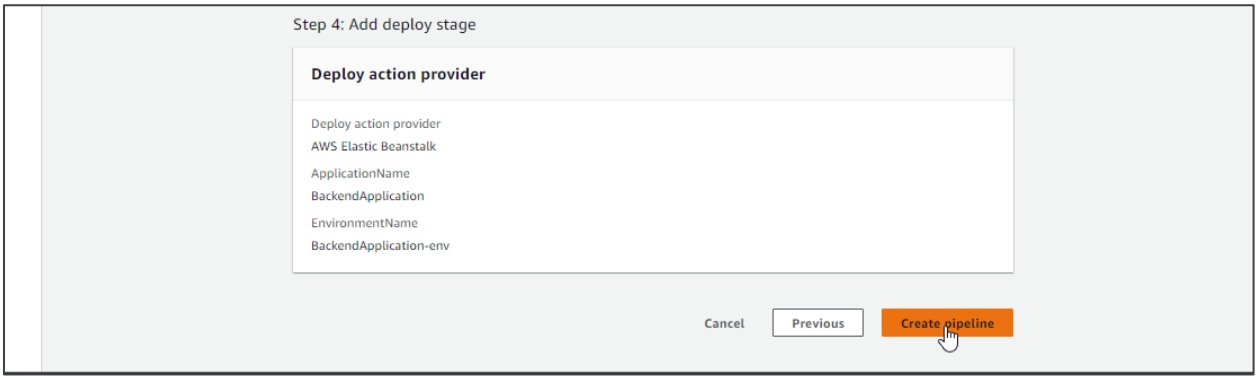


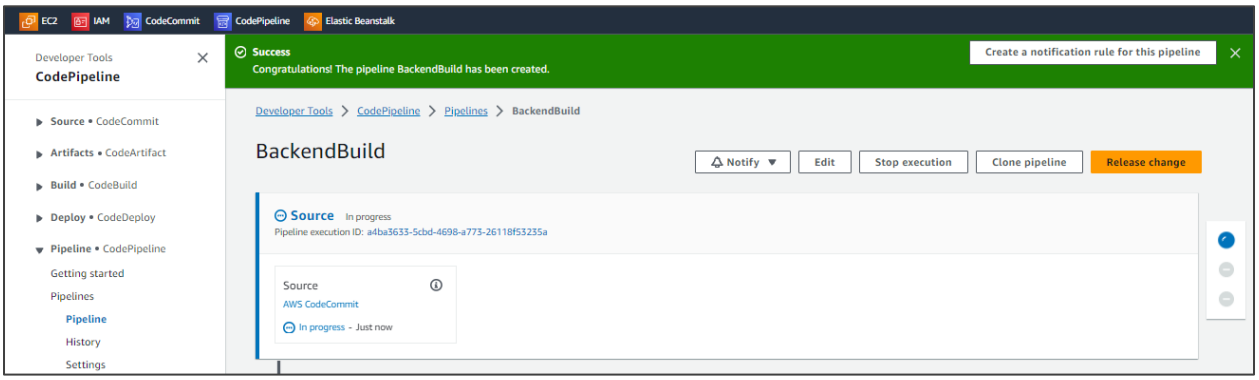
5.16 Configure the deployment of the backend Node.js application per the below screenshot:



5.17 Review the configurations and click on **Create pipeline**



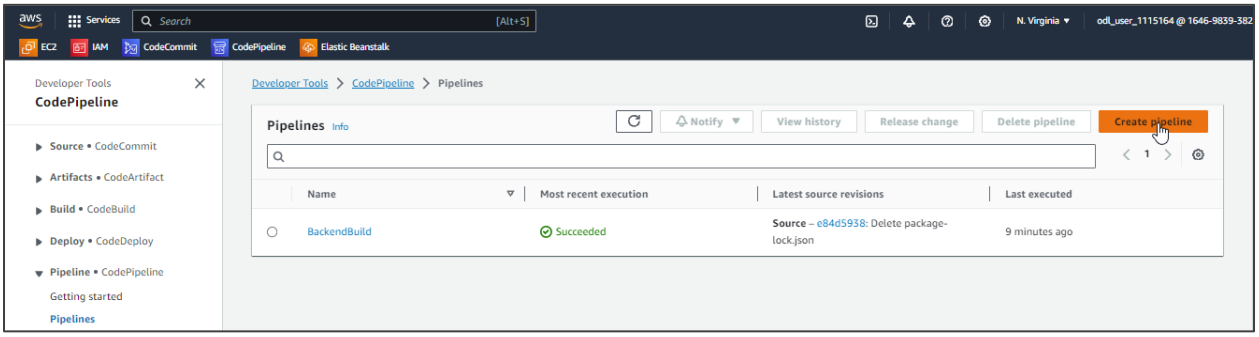




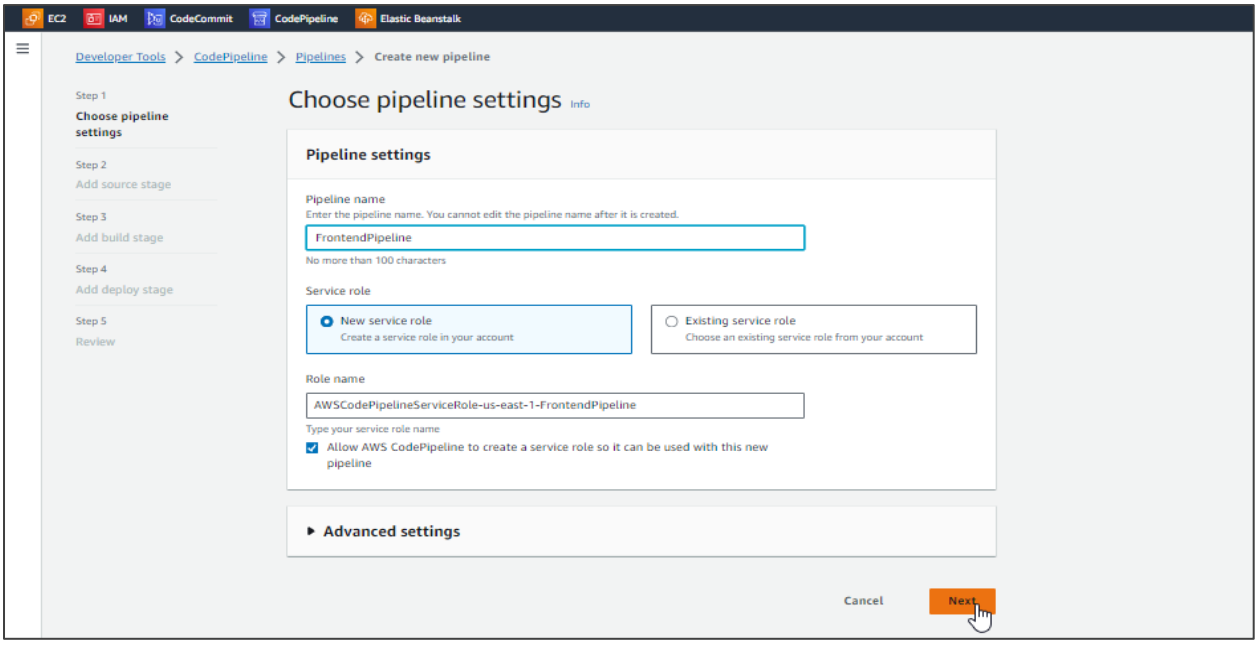
The backend application pipeline is successfully created.

Step 6: Set up the CodePipeline for the frontend application

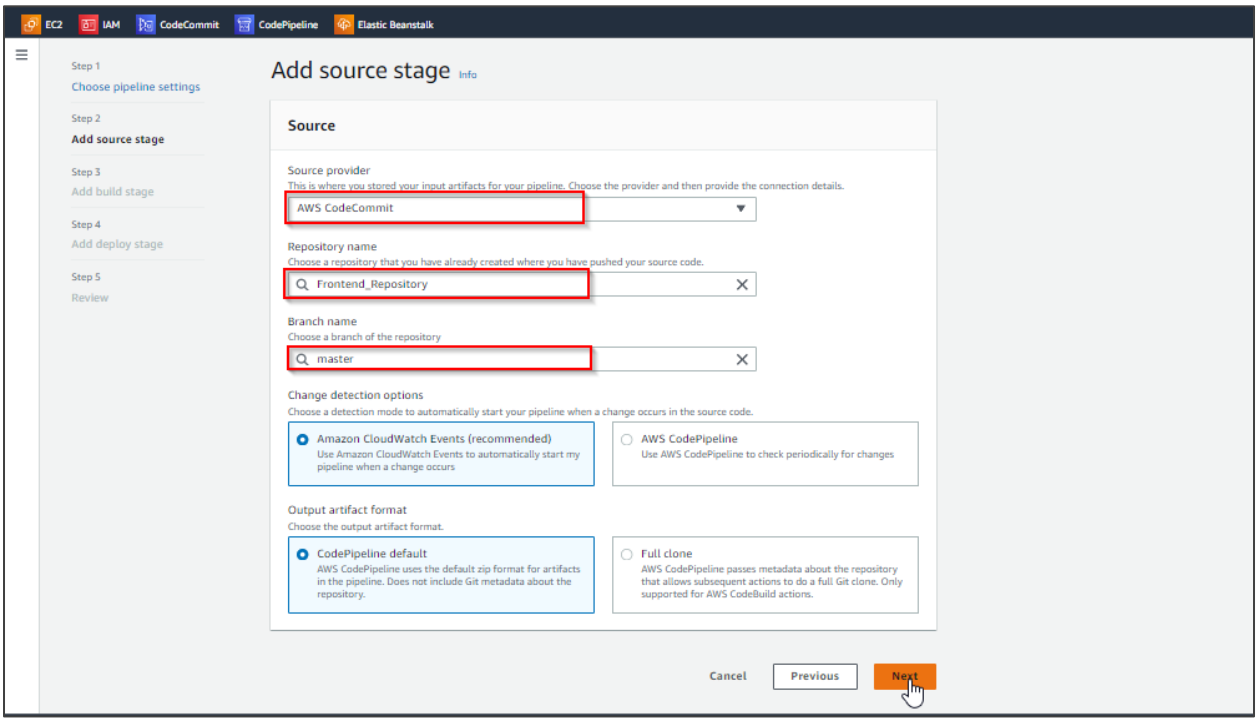
6.1 Click on **Create pipeline** to set up the frontend application pipeline



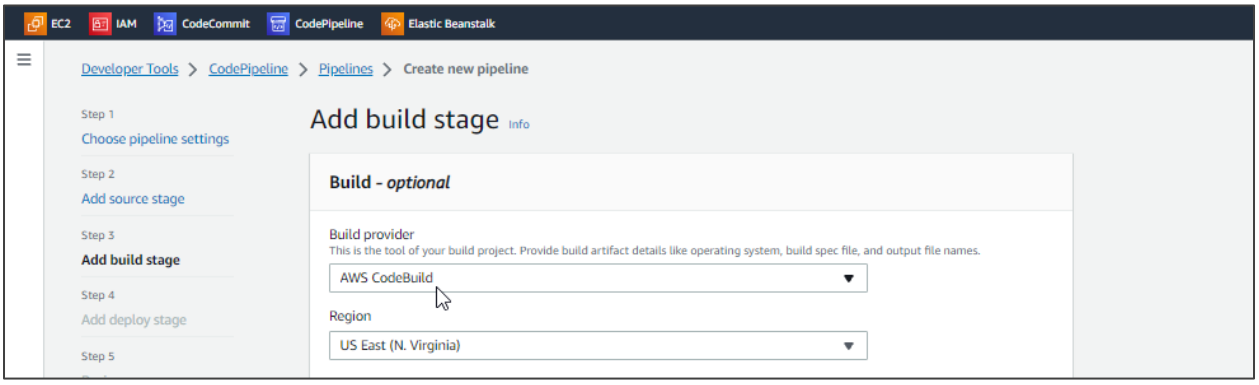
6.2 Enter a suitable name for the frontend pipeline, keep all the default configurations, and click on **Next**



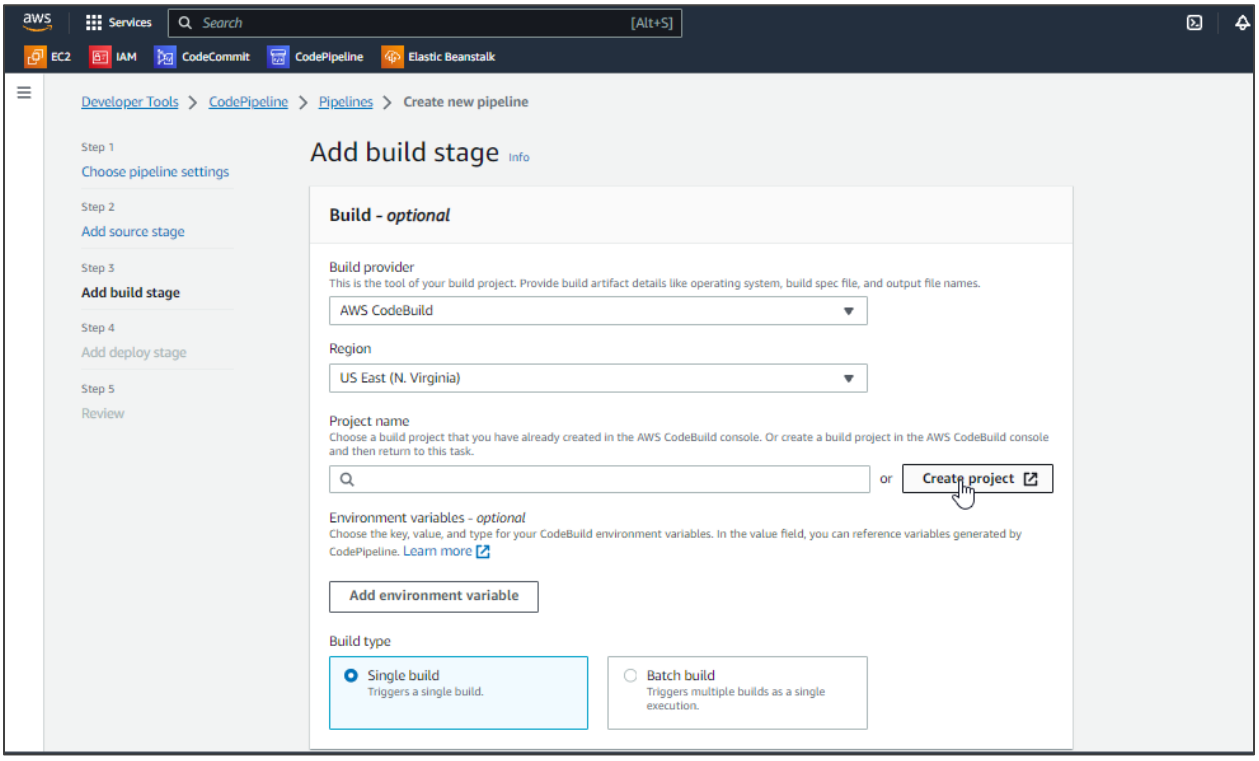
6.3 In the **Add source stage** page, select **AWS CodeCommit** as the Source provider, specify the frontend repository name and branch, choose **Amazon CloudWatch Events** as the monitoring service, and then click on **Next**



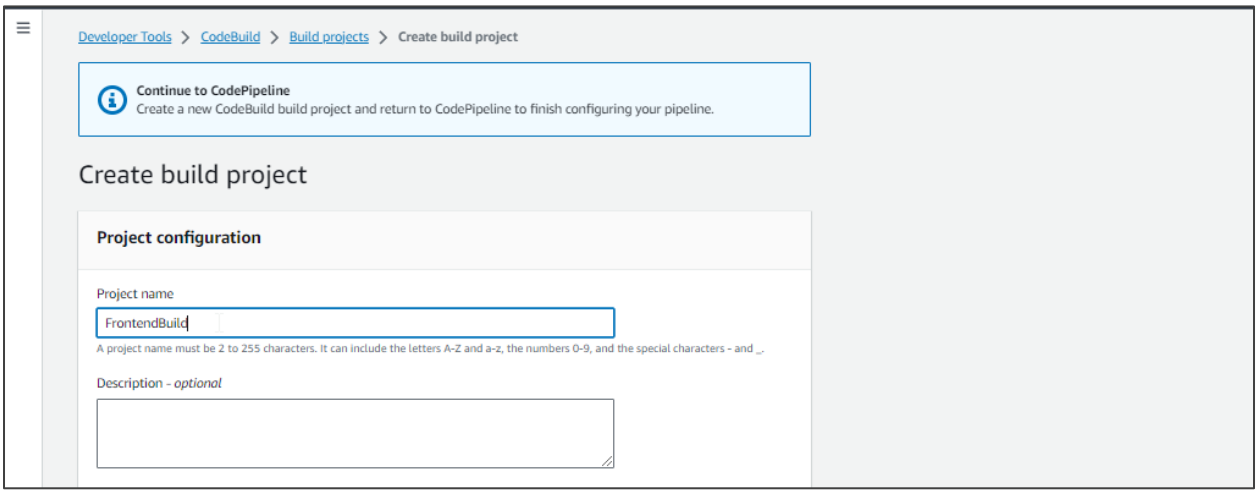
6.4 Select **AWS CodeBuild** as the Build provider



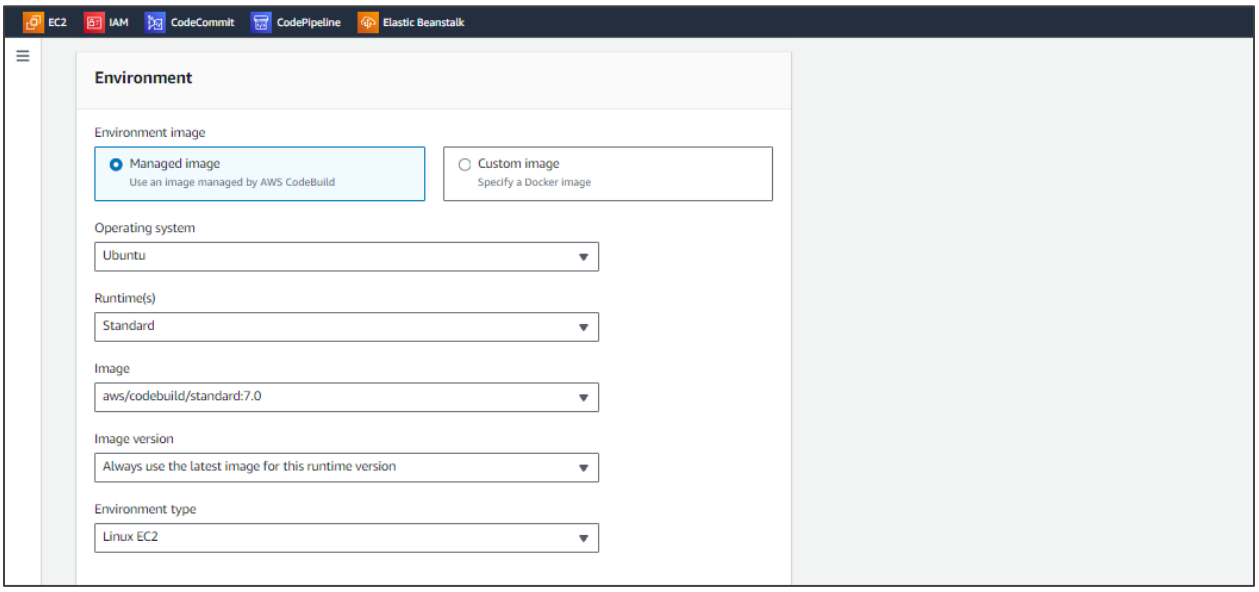
6.5 Click on **Create project** to create a new CodeBuild project, which will provide build, test, and execution-related steps



6.6 Enter a name for the CodeBuild project

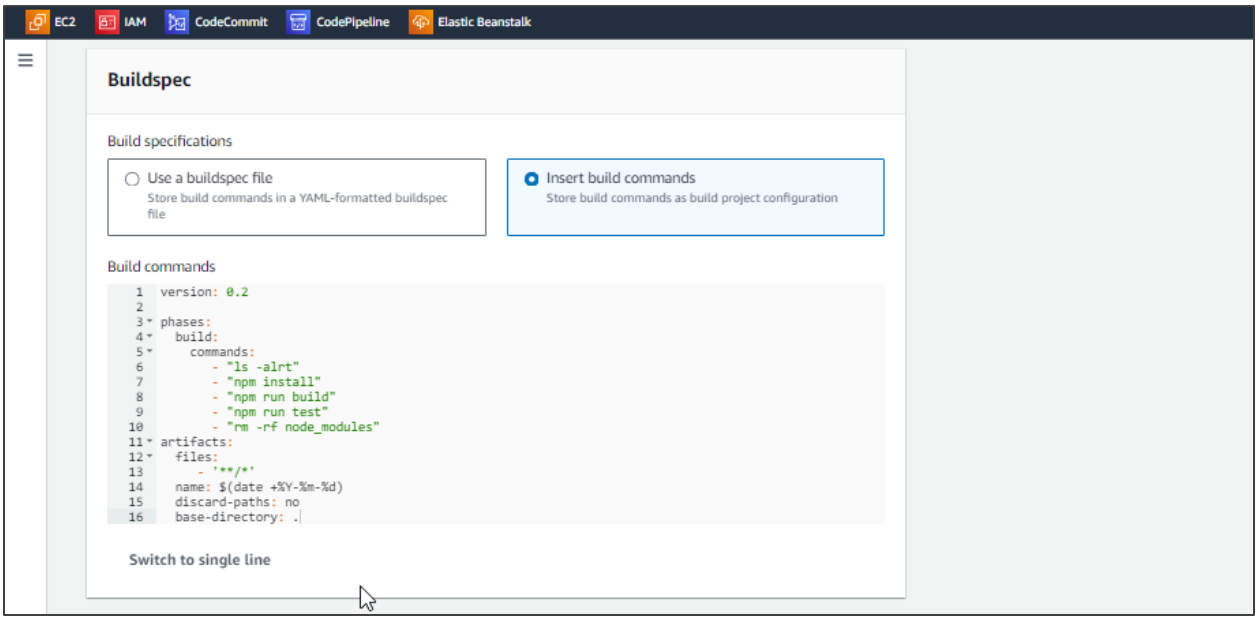


6.7 Under environment configurations, select **Ubuntu** as the Operating system, **Standard** as the Runtime(s), **aws/codebuild/standard:7.0** as Image, and **Linux EC2** as the environment type



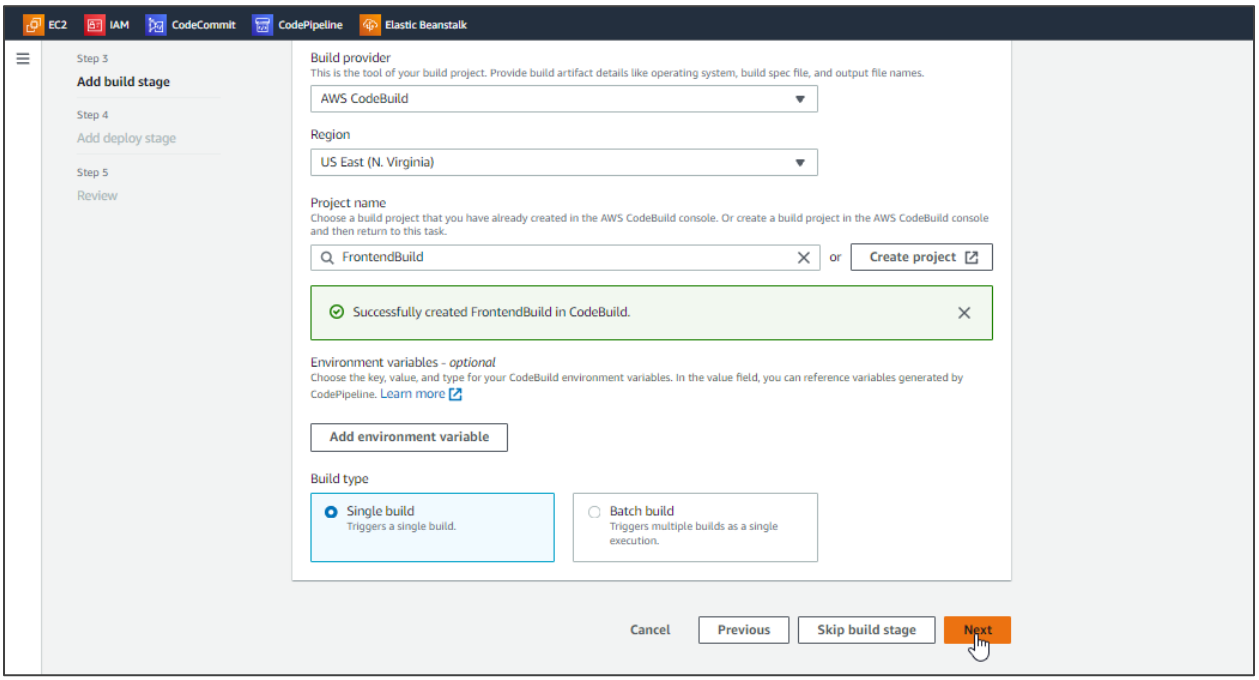
6.8 Under the **Buildspec** configuration, insert the following YAML code:

```
version: 0.2

phases:
  build:
    commands:
      - "ls -alrt"
      - "npm install"
      - "npm run build"
      - "npm run test"
      - "rm -rf node_modules"
artifacts:
  files:
    - '**/*'
  name: $(date +%Y-%m-%d)
  discard-paths: no
  base-directory: .
```

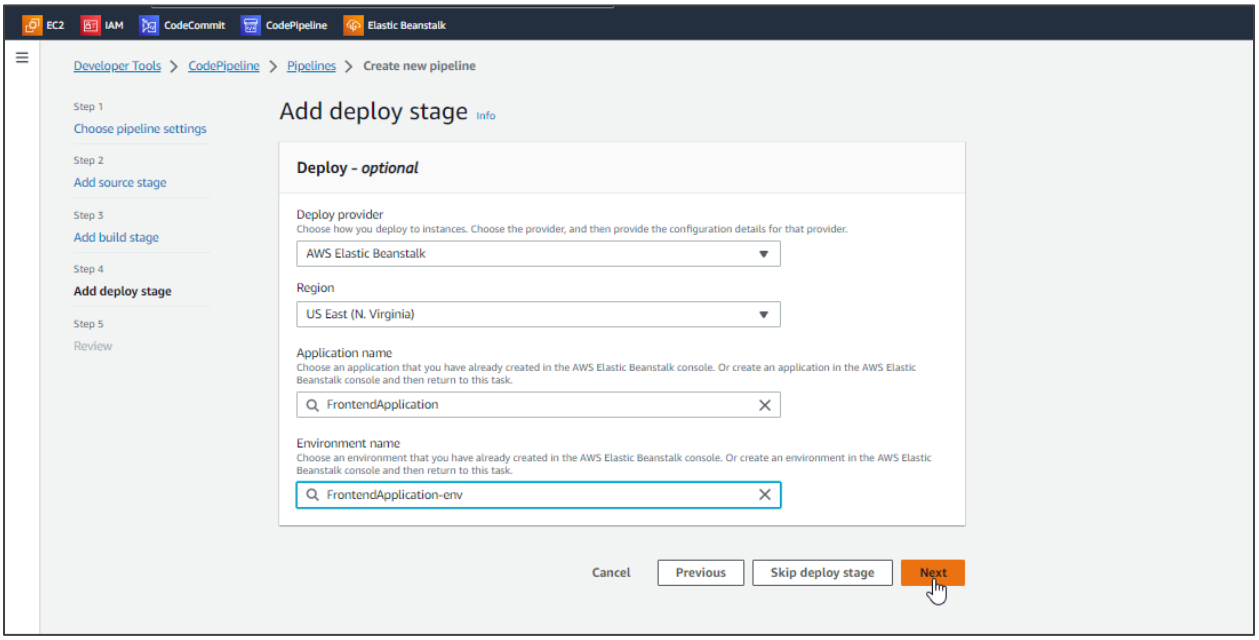


6.9 Next, click on **Continue to CodePipeline**

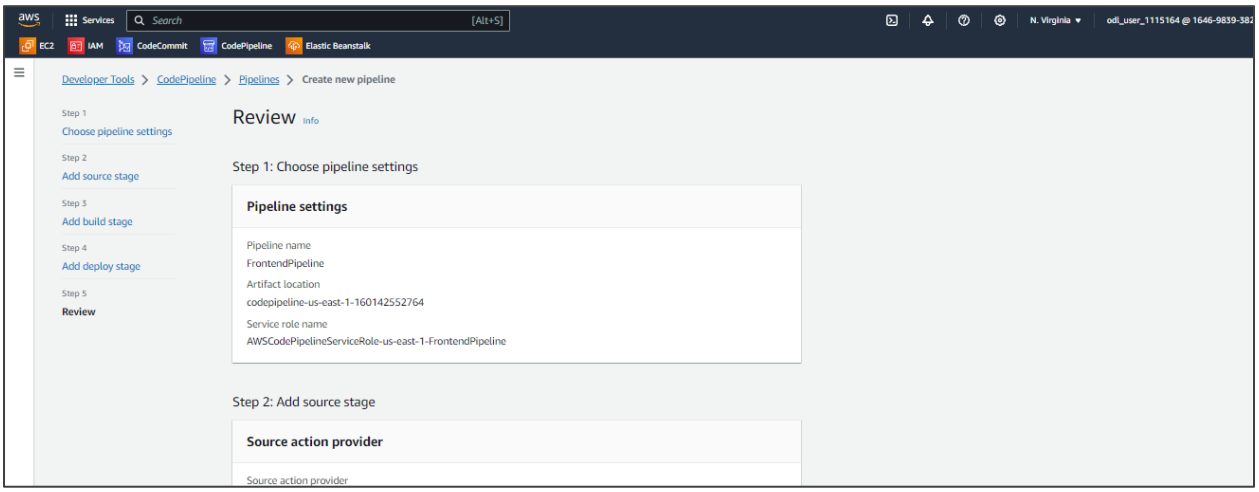
6.10 Once the CodeBuild project is created successfully, click on **Next.**

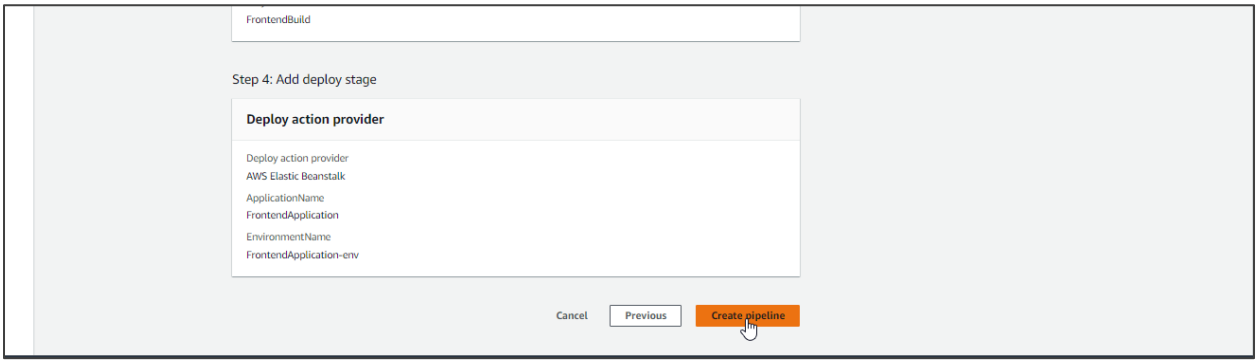


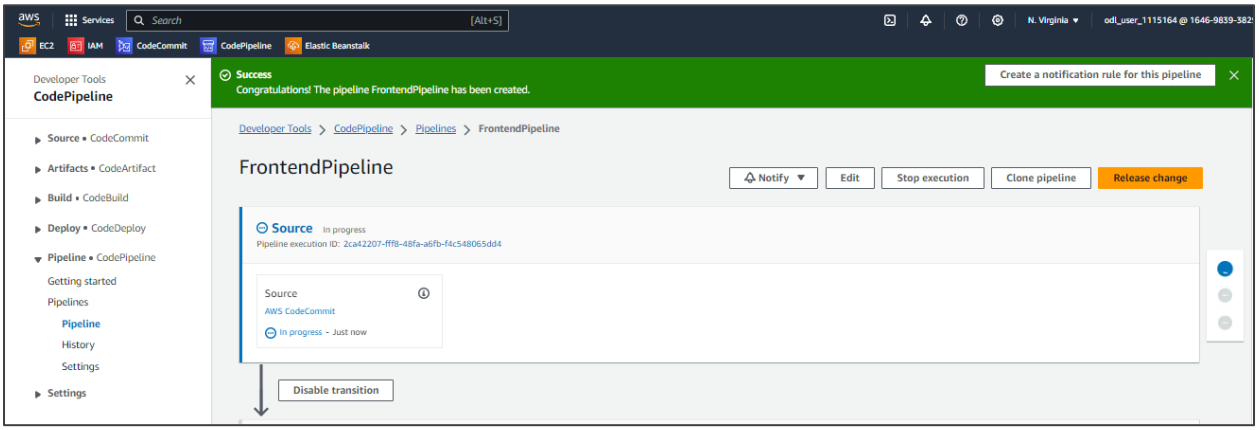
6.11 Configure the deployment of the frontend Node.js application per the below screenshot:



6.12 Review the configurations and click on **Create pipeline**



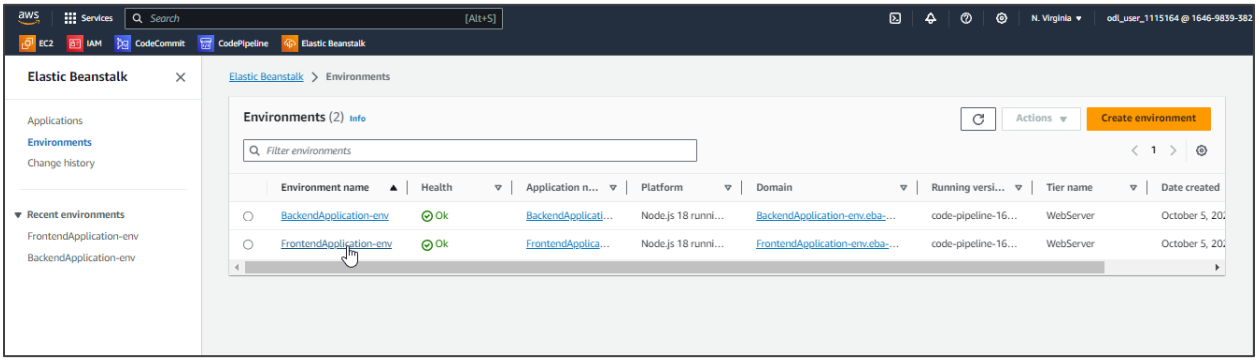




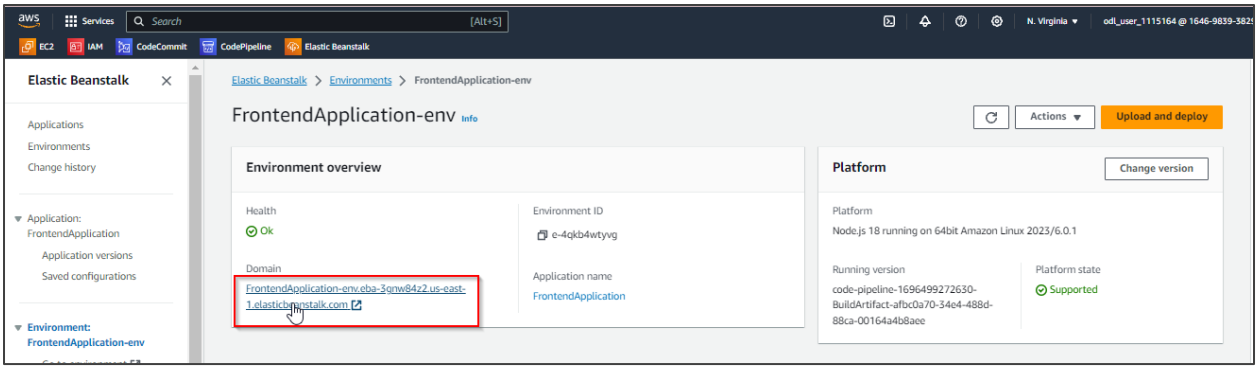
The pipeline for the frontend application is successfully created.

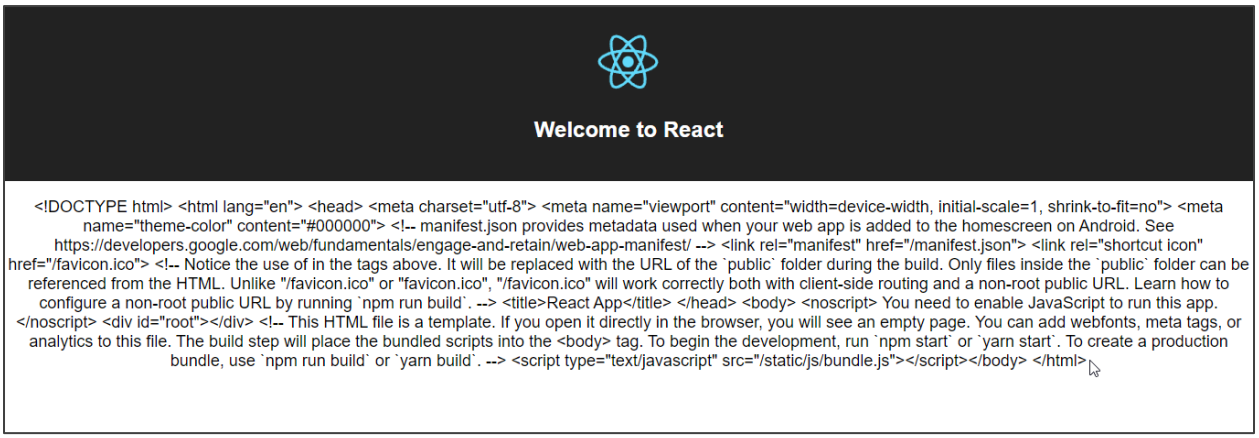
Step 7: Deploy the application

7.1 Go to the AWS Elastic Beanstalk console and click on **FrontendApplication-env**

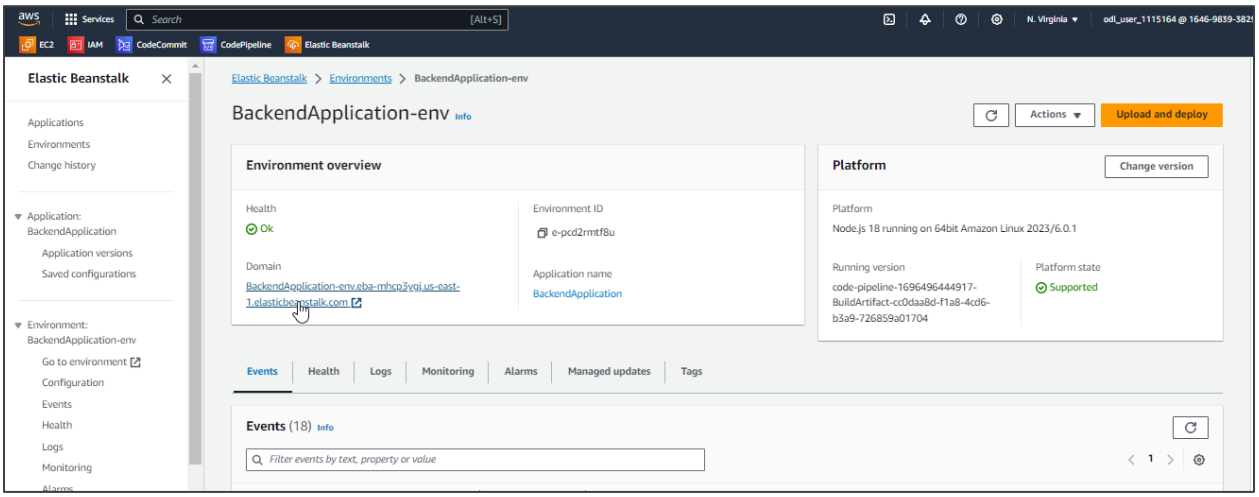


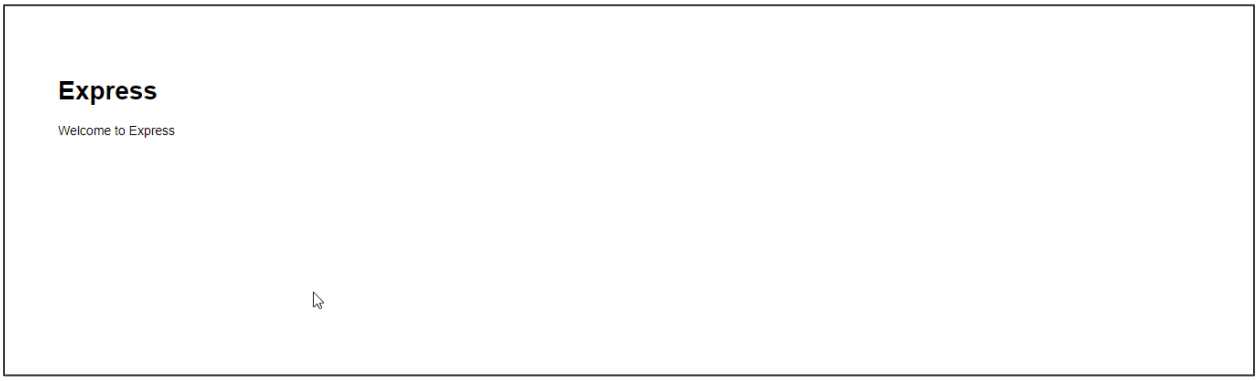
7.2 Click on the domain URL to validate the proper working of the frontend code





7.3 Similarly, validate the working of the backend code





By following these steps, you can successfully establish a CI/CD pipeline for a full-stack application deployment on AWS Elastic Beanstalk. This pipeline will streamline the development and deployment processes, ensuring efficient and consistent application updates.